Өмнөговь Технологийн Дээд Сургууль
Мэдээллийн Технологийн Тэнхим

М.Түвшинжаргал

Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

Бакалаврын төгсөлтийн ажил

Даланзадгад хот

Өмнөговь Технологийн Дээд Сургууль
Мэдээллийн Технологийн Тэнхим

Мэдээллийн технологийн тэнхим

Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

Мэргэжлийн индекс: D061304
Мэргэжил: Мэдээллийн технологи
Удирдагч: Доктор Д.Эрдэнэтуяа
Зөвлөгч:
Гүйцэтгэгч: М.Түвшинжаргал

Даланзадгад хот
2026 он

Батлав. Мэдээллийн технологийн тэнхимын эрхлэгч:

..... //

Удирдагч: ..... /Доктор Д.Эрдэнэтуяа/

ТӨГСӨЛТИЙН АЖЛЫН ТӨЛӨВЛӨГӨӨ

Төгсөлтийн ажлын сэдэв:
Монгол: Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт
Англи: Design and Implementation of a Three-Tier Sports Hall Booking System Based on Cloud Environment

Төслийн зорилго: Өмнөговь аймгийн Даланзадгад хотын спорт заалнуудын цаг захиалгыг цахимжуулах, хэрэглэгчдэд хялбар, хурдан захиалга хийх боломж олгох Flutter мобайл аппликейшн хөгжүүлэх.

Оюутны овог нэр, код: М.Түвшинжаргал/B221940619/

№	Ажлын бүлэг, хэсгийн нэр	эзлэх хувь	дуусах хугацаа
Бүлэг №1. Оршил ба судалгааны үндэслэл			
1	1.1 Судалгааны үндэслэл, асуудлын тодорхойлолт 1.2 Зорилго, зорилтууд 1.3 Ажлын ач холбогдол, хамрах хүрээ	20%	III.13
Бүлэг №1. Онол ба технологийн судалгаа			
2	2.1 Мобайл апп хөгжүүлэлтийн орчин үеийн арга хандлага 2.2 Flutter фреймворк ба Dart хэл 2.3 Firebase бэкэнд үйлчилгээ 2.4 Цаг захиалгын системийн онолын үндэс	30%	IV.10
Бүлэг №3. Системийн зохион байгуулалт ба хэрэгжүүлэлт			
3	3.1 Системийн шаардлагын дүн шинжилгээ 3.2 Системийн архитектур 3.3 Өгөгдлийн сангийн зохион байгуулалт 3.4 Flutter апп-ын хэрэгжүүлэлт 3.5 Туршилт ба үр дүнгийн үнэлгээ	50%	V.22
Ерөнхий дүгнэлт			

Төлөвлөгөөг боловсруулсан оюутан: ..... /М.Түвшинжаргал/

ТӨГСӨЛТИЙН АЖЛЫН ЯВЦ

№	Хийж гүйцэтгэсэн ажил	Биелсэн хугацаа	Удирдагчийн гарын үсэг
1	Бүлэг 1: Оршил, судалгааны үндэслэл боловсруулсан	2026-03-13	
2	Бүлэг 1: Онол ба технологийн судалгаа хийсэн	2026-04-10	
3	Бүлэг 3: Систем зохион байгуулж, Flutter апп хөгжүүлсэн	2026-05-15	
4	Туршилт хийж, дүгнэлт боловсруулсан	2026-05-22	

Ажлын товч дүгнэлт

.....

.....

.....

.....

.....

.....

.....

Удирдагч: ..... /Доктор Д.Эрдэнэтуяа/

ЗӨВШӨӨРӨЛ

Оюутан М.Түвшинжаргалгын бичсэн төгсөлтийн ажлыг УШК-д хамгаалуулахаар тодорхойлов.

Салбарын эрхлэгч: ..... //

Зохиогчийн эрх хамгаалал

Миний бие М.Түвшинжаргал, Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт сэдэвт энэ ажил нь минийх бөгөөд дараахыг нотолж байна. Үүнд:

- Горилогч энэ ажлыг тус сургуулиас боловсролын зэрэг авахаар бүхэлд нь буюу голлон хийсэн болно.
- Энэ ажлын аль нэг хэсгийг тус сургуульд эсвэл өөр байгууллагад боловсролын зэрэг, мэргэшил авахаар өмнө нь илгээсэн бол түүнийгээ тодорхой заасан болно.
- Бусад хүмүүсийн хэвлүүлсэн ажлаас зөвлөгөө авсан бол түүнийгээ үндэслэсэн болно.
- Бусад хүмүүсийн ажлаас ишлэл хийхдээ эх үүсвэрийг нь заасан болно.
- Миний ажилд тусалсан голлох бүх эх үүсвэрт талархаж байна.
- Ажлыг бусадтай хамтарсан бол алийг нь бусад хүмүүс хийсэн болохыг тодорхой заасан болно.

Гарын үсэг: _____

Огноо: _____

Өмнөговь Технологийн Дээд Сургууль
Мэдээллийн Технологийн Тэнхим

Хураангуй

Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

М.Түвшинжаргал

Түлхүүр үгс: *Flutter, Firebase, Node.js, Docker, AWS EC2, GitHub Actions, AI чатбот, цаг захиалга, спорт заал, үүлэн орчин, Даланзадгад*

Энэхүү бакалаврын төгсөлтийн ажлын зорилго нь Өмнөговь аймгийн Даланзадгад хотын таван спорт байгууламж — **Өмнөговь аймгийн Спорт цогцолбор** (спорт цогцолбор) болон **Говийн Арена**, **1-р болон 2-р хорооны Спорт заал**, **Даланзадгад Волейболын клуб заал** (хот айлын заалнууд) — -д зориулсан үүлэн орчинд суурилсан цаг захиалгын мобайл аппликейшн хөгжүүлэх явдал юм.

Систем нь **Flutter** (Android/iOS), **Node.js/Express** бэкэнд, **Firebase** (Auth, Firestore, Cloud Functions), **Docker/AWS EC2** дэд бүтэц болон **AI чатбот** (Claude Haiku, Groq Llama) гэсэн таван бүрэлдэхүүнтэй гурван давхаргат архитектурт суурилна. Бэкэнд нь **AWS EC2** (ap-southeast-1) дээр Docker контейнераар ажиллаж, **GitHub Actions** CI/CD хоолойгоор автоматаар байршуулагддаг.

Үндсэн функцүүд: email/нууц үгийн нэвтрэлт (Nodemailer 6 оронтой кодын баталгаажуулалттай), цаг захиалах болон цуцлах (DBK-xxxxx кодтой), байгууллагуудын тогтмол захиалга (`fixed_bookings`), 1 цагийн өмнөх локал мэдэгдэл, AI чатбот.

Функциональ туршилтаар 20 тест тохиолдол бүгд амжилттай давж, хэрэглэхэд хялбар байдлын нийт оноо **4.3/5.0** болсон.

Keywords: *Flutter, Firebase, Node.js, Docker, AWS EC2, GitHub Actions, AI chatbot, booking system, sports hall, cloud, Dalanzadgad*

This bachelor's thesis develops a cloud-based three-tier sports hall booking mobile application for Dalanzadgad city, Ömnögovi Province. The system comprises a Flutter mobile client (Android/iOS), a Node.js/Express backend on AWS EC2 via Docker, Firebase services (Authentication, Firestore, Cloud Functions), and dual AI chatbot (Anthropic Claude Haiku 4.5 / Groq Llama 3.1-8b). Automated deployment is handled by GitHub Actions CI/CD. All 20 test cases passed with a usability score of 4.3/5.0.

Талархал

Энэхүү бакалаврын төгсөлтийн ажлыг гүйцэтгэхэд чиглэл заагч, зөвлөгөө өгч, тасралтгүй дэмжиж ирсэн удирдагч багш **Доктор Д.Эрдэнэтуяа** багш танд гүн талархал илэрхийлье.

Дөрвөн жилийн турш мэдлэг, чадвар, судалгааны арга барилыг эзэмшихэд тусалсан **Өмнөговь Технологийн Дээд Сургуулийн** бүх багш нарт талархал илэрхийлж байна.

Мөн судалгааны явцад туршилтад оролцож, санал хүсэлтээ өгсөн **Өмнөговь аймгийн Даланзадгад хотын спорт заалнуудын** захиралууд болон ажилчдад талархал илэрхийлье.

Эцэст нь, тасралтгүй урам өгч, дэмжиж ирсэн гэр бүлийнхэндээ болон хамт сурдаг нөхдөдөө чин сэтгэлийн талархлаа илэрхийлье.

Товчилсон үгс

UI	User Interface (Хэрэглэгчийн интерфэйс)
UX	User Experience (Хэрэглэгчийн туршлага)
API	Application Programming Interface (Аппликейшн програмчлалын интерфэйс)
SDK	Software Development Kit (Програм хангамжийн хөгжүүлэлтийн хэрэгсэл)
IDE	Integrated Development Environment (Нэгдсэн хөгжүүлэлтийн орчин)
OS	Operating System (Үйлдлийн систем)
iOS	iPhone Operating System (Apple-ийн мобайл үйлдлийн систем)
APK	Android Package Kit (Android апп-ын суулгац файл)
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation (Өгөгдөл солилцооны формат)
JWT	JSON Web Token (Нэвтрэлтийн токен формат)
REST	Representational State Transfer
MVC	Model-View-Controller (Загвар-Харагдац-Удирдагч)
DB	Database (Өгөгдлийн сан)
SQL	Structured Query Language
NoSQL	Not only SQL (Уламжлалт бус өгөгдлийн сан)
OTP	One-Time Password (Нэг удаагийн нууц үг)
FCM	Firebase Cloud Messaging (Firebase-ийн мэдэгдлийн үйлчилгээ)
CI/CD	Continuous Integration / Continuous Deployment (Тасралтгүй нэгтгэлт / байршуулалт)
AWS	Amazon Web Services (Амазоны үүлэн үйлчилгээ)
EC2	Elastic Compute Cloud (AWS-ийн виртуал сервер үйлчилгээ)
ECR	Elastic Container Registry (AWS-ийн Docker дүрс хадгалах сан)
VPC	Virtual Private Cloud (Виртуал хувийн сүлжээ)
IAM	Identity and Access Management (AWS нэвтрэлт, эрх удирдлага)
AI	Artificial Intelligence (Хиймэл оюун ухаан)
LLM	Large Language Model (Том хэлний загвар)
SaaS	Software as a Service (Үйлчилгээ хэлбэрийн програм хангамж)
BaaS	Backend as a Service (Үйлчилгээ хэлбэрийн бэкэнд)
UML	Unified Modeling Language (Нэгдсэн загварчлалын хэл)
QR	Quick Response (Шуурхай хариу кодчилол)

Гарчиг

Зохиогчийн эрх хамгаалал **i**

Хураангуй **ii**

Талархал **iii**

Товчилсон үгс **iv**

Удиртгал **viii**

1 ОНОЛ БА ТЕХНОЛОГИЙН СУДАЛГАА **1**
1.1 Мобайл хөгжүүлэлтийн технологи 1
1.2 Бэкэнд хөгжүүлэлтийн технологи 2
1.3 Үүлэн орчин ба автоматжуулалт 3

2 СИСТЕМИЙН ЗАГВАРЧЛАЛ БА ХӨГЖҮҮЛЭЛТ **6**
2.1 Системийн шаардлага ба загварчлал 6
2.1.1 Функциональ болон функциональ бус шаардлага 6
2.1.2 UML Диаграммууд 6
2.1.3 Системийн архитектур 7
2.2 Frontend хөгжүүлэлт 7
2.2.1 Дизайн систем ба навигаци 7
2.2.2 Нэвтрэлт болон цаг захиалгын хэрэгжүүлэлт 11
2.3 Backend хөгжүүлэлт 13
2.3.1 Express.js API ба email баталгаажуулалт 13
2.3.2 AI чатбот болон cron ажил 15
2.4 Өгөгдлийн сан 16

3 ХЭРЭГЖҮҮЛЭЛТ **18**
3.1 Үүлэн байршуулалт 18
3.1.1 Docker контейнержуулалт ба AWS EC2 18
3.1.2 GitHub Actions CI/CD хоолой 19
3.2 Туршилт ба үнэлгээ 20
3.2.1 Функциональ туршилт 20
3.2.2 Гүйцэтгэлийн хэмжилт ба хэрэглэхэд хялбар байдал 21
3.2.3 Туршилтын нийт дүн 22

Дүгнэлт **24**

A Аппликейшний дэлгэцийн зураглал **26**

B Үндсэн тохиргооны файлууд **29**

Ном зүй **32**

Зургийн жагсаалт

2.1 Use Case Diagram — хэрэглэгч болон Admin-ийн харилцаа Эх: Зохиогчийн боловсруулалт . . . 7
2.2 Цаг захиалах үйл ажиллагааны диаграмм (Activity Diagram) Эх: Зохиогчийн боловсруулалт . . . 8
2.3 Захиалга үүсгэх дараалын диаграмм (Sequence Diagram) Эх: Зохиогчийн боловсруулалт . . . 9
2.4 Email баталгаажуулалтын дараалын диаграмм Эх: Зохиогчийн боловсруулалт . . . 10
2.5 Системийн гурван давхаргат үүлэн архитектур Эх: Зохиогчийн боловсруулалт . . . 11
2.6 Flutter аппликейшний давхаргат архитектур Эх: Зохиогчийн боловсруулалт 12
2.7 Email/нууц үгийн нэвтрэлт болон баталгаажуулалтын дэлгэцүүд Эх: Зохиогчийн зураг авалт . . . 13
2.8 Нүүр дэлгэц, цаг захиалах болон захиалгын жагсаалтын дэлгэцүүд Эх: Зохиогчийн зураг авалт . . . 14
2.9 Захиалгын сануулга болон AI туслахын дэлгэцүүд Эх: Зохиогчийн зураг авалт . . . 15
2.10 Firestore collection-уудын схем диаграмм Эх: Зохиогчийн боловсруулалт . 17

3.1 Үүлэн байршуулалтын архитектур — AWS EC2, Firebase, CI/CD Эх: Зохиогчийн боловсруулалт . . . 18
3.2 GitHub Actions CI/CD хоолойн дэлгэрэнгүй дараалын диаграмм Эх: Зохиогчийн боловсруулалт . . . 20

A.1 Нэвтрэх дэлгэц (зүүн) ба бүртгэлийн дэлгэц (баруун) . . . 26
A.2 Нүүр дэлгэц — таван спорт заалны жагсаалт . . . 27
A.3 Цагийн хуваарийн дэлгэц (зүүн) ба захиалга баталгаажуулах дэлгэц (баруун) . . . 28
A.4 Захиалгын жагсаалтын дэлгэц (зүүн) ба AI чатботын дэлгэц (баруун) 28

Хүснэгтийн жагсаалт

1 Одоогийн цаг захиалгын асуудлууд ба шийдэл viii

1.1 Мобайл аппликейшн хөгжүүлэлтийн арга хандлагуудын харьцуулалт 1
1.2 Node.js/Express REST API endpoint-уудын жагсаалт 3
1.3 Ашигласан технологийн стекийн нэгтгэсэн хүснэгт 5

2.1 Функциональ бус шаардлагуудын хүснэгт 6

3.1 Функциональ туршилтын тохиолдлууд 21
3.2 Нэгтгэлийн туршилтын үр дүн 22
3.3 Системийн хариу цагийн хэмжилт (секунд) болон шаардлагатай харьцуулалт 22
3.4 Хэрэглэхэд хялбар байдлын үнэлгээний үр дүн 23
3.5 Туршилтын нийт үр дүн — шаардлагатай харьцуулалт 23
3.6 Зорилтуудын биелэлт 25

УДИРТГАЛ

Сэдэв сонгосон үндэслэл

Мэдээллийн технологийн хурдацтай хөгжил нь орчин үеийн иргэдийн өдөр тутмын амьдралыг ихээхэн өөрчилж байна. Цахим захиалга, онлайн үйлчилгээ нь дэлхийн өнцөг булан бүрт нэвтэрч, хүмүүсийн цаг хугацааг хэмнэж, үйлчилгээний чанарыг дээшлүүлж байна. Statista судалгааны байгууллагын 2024 оны тайланд дурдснаар дэлхийн мобайл аппликейшний зах зээлийн хэмжээ 2028 он гэхэд 756 тэрбум ам.долларт хүрнэ гэж таамаглаж байна [1]. Гэвч Монгол улсын орон нутгийн хотуудад, тухайлбал Өмнөговь аймгийн Даланзадгад хотод, уламжлалт үйлчилгээний хэлбэр давамгайлсаар байна.

Даланзадгад хотод өнөөдрийн байдлаар таван спорт заал болон тамирын цогцолбор үйл ажиллагаа явуулж байна: Өмнөговь аймгийн Спорт цогцолбор, Говийн Арена Спорт заал, 1-р хорооны Спорт заал, 2-р хорооны Спорт заал, Даланзадгад Волейболын клуб заал. Эдгээр байгууламж нь аймгийн иргэдийн биеийн тамирын үйл ажиллагааны гол бааз болдог.

Иргэд эдгээр заалд цаг захиалахдаа тулгамдаж буй асуудлууд болон тэдгээрийн шийдлийг доорх хүснэгтэд үзүүлэв.

Хүснэгт 1: Одоогийн цаг захиалгын асуудлууд ба шийдэл

#	Тулгамдаж буй асуудал	Системийн шийдэл
1	Утсаар захиалахад шугам завгүй байх, буруу ойлголцол үүсэх	Онлайн захиалга — цагийн хуваарь бодит цагаар харагдах
2	Биечлэн очиж захиалахад цаг, зардал зарцуулагдах	Хаанаас ч гэсэн мобайл аппаар захиалах боломж
3	Боломжтой цагийн мэдээлэл тодорхойгүй байх	Firestore бодит цагийн шинэчлэлтээр цагийн хуваарь нэн даруй харагдах
4	Захиалгаа цуцлах, өөрчлөх боломж хязгаарлагдмал байх	Апп дотроос нэг дарахад захиалга цуцлах функц
5	Захиалгын баталгаажуулалт, сануулга авах боломжгүй байх	DBK-xxxxx захиалгын код болон 1 цагийн өмнөх локал мэдэгдэл

Эх: Зохиогчийн боловсруулалт

Эдгээр асуудлыг шийдвэрлэхийн тулд орчин үеийн мобайл болон үүлэн технологийг ашиглан цаг захиалгын систем хөгжүүлэх нь чухал шаардлага болж байна. **Flutter** фреймворк нь нэг удаа бичиж Android болон iOS платформд нэгэн зэрэг гаргах боломж олгодог тул хөгжүүлэлтийн зардал, хугацааг мэдэгдэхүйц бууруулдаг. **Firebase** нь Google-ийн үүлэн бэкэнд үйлчилгээ бөгөөд бодит цагийн өгөгдөл солилцоо, найдвартай нэвтрэлт, тасралтгүй ажиллагааг хангадаг. **Node.js/Express** бэкэнд нь **Docker** контейнержуулалтаар **AWS EC2** үүлэн серверт байршиж, **GitHub Actions** CI/CD хоолойгоор автоматаар нийтлэгддэг тул систем нь бодитоор үүлэн орчинд ажилладаг.

Зорилго

Энэхүү ажлын зорилго нь Өмнөговь аймгийн Даланзадгад хотын таван спорт байгууламжийн цаг захиалгыг цахимжуулж, хэрэглэгчдэд хялбар, хурдан, ил тод байдлаар захиалга хийх боломж олгох **Flutter** мобайл аппликейшн болон **Node.js/Express**

бэкэнд бүхий гурван давхаргат системийг үүлэн орчинд хөгжүүлж, байршуулах явдал юм.

Зорилт

Уг зорилгод хүрэхийн тулд дараах зорилтуудыг дэвшүүлж байна. Нэгдүгээрт, Даланзадгад хотын спорт заалнуудын цаг захиалгын одоогийн нөхцөл байдал, иргэдийн хэрэгцээг судлаж, тулгамдаж буй асуудлуудыг тогтооно. Хоёрдугаарт, цаг захиалгын мобайл аппликейшний функциональ болон функциональ бус шаардлагын дүн шинжилгээ хийж, Use Case Diagram, Activity Diagram, Sequence Diagram боловсруулна. Гуравдугаарт, Flutter болон Firebase технологийг ашиглан гурван давхаргат системийн архитектур зохион байгуулна. Дөрөвдүгээрт, Material Design 3 стандартад нийцсэн мобайл аппликейшн хэрэгжүүлнэ. Тавдугаарт, Node.js/Express REST API бэкэнд хөгжүүлж, Docker контейнержуулалтаар AWS EC2-д байршуулна. Зургадугаарт, GitHub Actions CI/CD хоолойгоор бэкэндийн автоматжуулсан байршуулалтыг хэрэгжүүлнэ. Долоодугаарт, функциональ, нэгтгэлийн болон гүйцэтгэлийн туршилт хийж, хэрэглэхэд хялбар байдлыг үнэлнэ.

Судлагдсан байдал

Спорт заал болон үйлчилгээний цаг захиалгын мобайл аппликейшн хөгжүүлэлтийн чиглэлд олон улсын болон дотоодын судлаачид тодорхой хэмжээний ажил хийсэн байна.

Монгол улсын хэмжээнд eZaal (DAZO LLC) нь спортын байгууламжийн захиалгыг нэгтгэсэн томоохон платформ бөгөөд Google Play дээр 10,000 гаруй удаа татагдсан [2]. Kassim et al. (2022) судалгаанд Flutter-г ашиглан sport facility booking систем хөгжүүлсэн туршлагыг дүрсэлсэн бөгөөд Firebase-тэй хослуулсан нь хөгжүүлэлтийн хугацааг 40 хувиар бууруулсан гэж тэмдэглэжээ [3].

Монголын эрдэм шинжилгээний хэмжээнд Амарсайханы Билгүүн [4] мобайл аппликейшн хөгжүүлэлтийн аргачлалын асуудлыг судалсан бол Т.Амартүвшин, Б.Зоригтбаатар нар [5] мобайл апп хөгжүүлэлтийн онолын үндсийг тодорхойлсон байна. Гэхдээ тухайлан үүлэн орчинд суурилсан, гурван давхаргат архитектур бүхий спорт заалны цаг захиалгын тусгай системийн талаарх судалгаа орон нутгийн хэмжээнд хийгдээгүй байна. Иймд энэхүү ажил нь орон нутгийн хэрэгцээнд нийцсэн, cross-platform, AI чатбот болон CI/CD автоматжуулалт бүхий орчин үеийн технологийн бүрэн стек ашигласан шинэлэг шийдлийг санал болгож байна.

Шинэлэг тал

Энэхүү ажлын шинэлэг тал нь дараахид оршино. Нэгдүгээрт, Flutter клиент, Node.js/Express бэкэнд (AWS EC2), Firebase/Google Cloud дэд бүтэц гэсэн тодорхой давхаргаар зохион байгуулсан гурван давхаргат үүлэн архитектурыг хэрэгжүүлсэн. Хоёрдугаарт, GitHub Actions ашиглан код шинэчлэгдэх бүрт Docker дүрс build хийж, AWS ECR-д push хийж, EC2-д автоматаар байршуулдаг 3–4 минутын CI/CD хоолой бий болгосон. Гуравдугаарт, Anthropic Claude Haiku 4.5 болон Groq Llama 3.1-8b загваруудыг нөөцлөлтийн горимоор нэгтгэсэн давхар горимын AI чатбот хэрэгжүүлсэн. Дөрөвдүгээрт, Даланзадгад хотын байгууллагуудын долоо хоногийн тогтмол цагийг удирдах тусгай `fixed_bookings` функц нэмсэн. Тавдугаарт, Монгол улсын орон нутгийн хотод анх удаа cross-platform, AI чатбот бүхий спорт заалны мобайл захиалгын систем хэрэгжүүлсэн.

Судалгааны хамрах хүрээ

Энэхүү ажлын хүрээнд иргэдэд зориулсан мобайл апп (Android 8.0+ болон iOS 13.0+), Firebase дээр суурилсан email/нууц үгийн нэвтрэлт (Nodemailer 6 оронтой кодын баталгаажуулалттай), Node.js/Express REST API бэкэнд (5 endpoint), Docker контейнержуулалт болон AWS EC2/ECR байршуулалт (ap-southeast-1), GitHub Actions CI/CD автоматжуулалт, AI чатботын үйлчилгээ (Claude Haiku 4.5 болон Groq Llama 3.1-8b), мөн локал мэдэгдлийн систем орно. Онлайн төлбөрийн систем (QPay, SocialPay), GPS байршлын нарийн функц болон администраторын дэлгэц нь энэ ажлын хамрах хүрээнд ороогүй болно.

Ач холбогдол

Энэхүү ажил нь Даланзадгад хотын иргэдэд спорт заал захиалах үйл явцыг хялбарчилж, цаг хугацааг хэмнэнэ. Спорт заалны ажилтнуудад захиалгын менежментийг автоматжуулж, ажлын ачааллыг бууруулна. Цахим захиалга нэвтэрснээр буруу ойлголцол болон шугам завгүй байхтай холбоотой асуудлуудыг арилгаж, захиалгын ил тод байдлыг хангана. Мөн энэ ажил нь орон нутгийн хотуудад цахим үйлчилгээ нэвтрүүлэх практик жишиг болж, цаашид ижил төрлийн шийдлүүдэд тулгуур болох ач холбогдолтой.

Ажлын бүтэц

Энэхүү бакалаврын төгсөлтийн ажил дараахи бүтэцтэй.

Удиртгалд судалгааны үндэслэл, зорилго, зорилт, судлагдсан байдал, шинэлэг тал, хамрах хүрээ, ач холбогдлыг тодорхойлсон.

1-р бүлэгт (Онол ба технологийн судалгаа) гурван тулгуур асуудлыг онолын үндэслэлтэйгээр авч үзсэн. 1.1 дэд бүлэгт мобайл хөгжүүлэлтийн технологи — Flutter, Dart, Firebase, Material Design 3 — яагаад сонгосон, хэрхэн ашигласан болохыг тайлбарласан. 1.2 дэд бүлэгт бэкэнд хөгжүүлэлтийн технологи — Node.js, Express, REST API, AI чатбот — -ийн онолын үндэс болон системд гүйцэтгэх үүргийг тодорхойлсон. 1.3 дэд бүлэгт үүлэн орчин ба автоматжуулалт — Docker, AWS EC2/ECR, GitHub Actions CI/CD — хэрэгтэй болсон шалтгаан, хэрэглэх аргачлалыг судалсан.

2-р бүлэгт (Системийн загварчлал ба хөгжүүлэлт) системийг зохион байгуулж, хэрэгжүүлсэн байдлыг дөрвөн дэд бүлгээр нарийвчлан тайлбарласан. 2.1 дэд бүлэгт функциональ болон функциональ бус шаардлагын дүн шинжилгээ, Use Case, Activity, Sequence диаграммуудыг боловсруулсан. 2.2 дэд бүлэгт Flutter аппликейшний дэлгэцүүд, навигаци, хэрэглэгчийн интерфейсийн хэрэгжүүлэлтийг харуулсан. 2.3 дэд бүлэгт Node.js/Express REST API болон email баталгаажуулалтын систем, AI чатботын хэрэгжүүлэлтийг тайлбарласан. 2.4 дэд бүлэгт Firestore өгөгдлийн сангийн бүтэц, collection-уудын загварыг харуулсан.

3-р бүлэгт (Хэрэгжүүлэлт) системийг бодит орчинд байршуулж, туршсан үр дүнг тайлбарласан. 3.1 дэд бүлэгт Docker контейнержуулалт, AWS EC2 байршуулалт, GitHub Actions CI/CD хоолойн хэрэгжүүлэлтийг дэлгэрэнгүй харуулсан. 3.2 дэд бүлэгт функциональ туршилт (шаардлагатай холбосон), нэгтгэлийн туршилт, гүйцэтгэлийн хэмжилт, хэрэглэхэд хялбар байдлын үнэлгээний үр дүнг нэгтгэсэн.

Дүгнэлтэд ажлын нийт үр дүн, хүрсэн зорилтуудын биелэлт болон цаашдын хөгжлийн чиглэлийг дүгнэсэн.

БҮЛЭГ 1

ОНОЛ БА ТЕХНОЛОГИЙН СУДАЛГАА

Даланзадгад хотын спорт заалны цаг захиалгын системийг хэрэгжүүлэхийн тулд технологийн сонголт нь дараах гурван шаардлагыг хангахад чиглэсэн: иргэдэд хаанаас ч нэвтрэх боломж олгох мобайл технологи; аюулгүй байдал болон бизнесийн логикийг тусад нь боловсруулах бэкэнд технологи; системийг тасралтгүй ажиллуулах, автоматаар шинэчлэх үүлэн дэд бүтэц. Энэ бүлэгт уг гурван шаардлагт нийцэх технологи бүрийн онолын үндэс, зориулалт болон яагаад тэр технологийг сонгосон шалтгааныг тайлбарлана.

1.1 Мобайл хөгжүүлэлтийн технологи

Спорт заал захиалах үйлчилгээг иргэдэд хялбар болгохын тулд мобайл аппликейшн зайлшгүй шаардлагатай болсон. Өнөөдөр хүн бүр ухаалаг гар утас ашиглаж байгаа тул апп нь аль ч цагт, аль ч газраас захиалга хийх боломж олгодог. Мобайл аппликейшн хөгжүүлэхэд гурван үндсэн арга хандлага байдаг: Native хандлагаар Android болон iOS-д тус тусад нь тусдаа хэлээр хоёр апп бичдэг; Hybrid хандлагаар вэб технологид тулгуурлан WebView-д ажиллуулдаг; Cross-platform хандлагаар нэг кодын баазаас хоёр платформд нэгэн зэрэг гаргадаг. Native хандлагыг сонгоход хоёр баг, хоёр дахин илүү хугацаа шаарддаг. Hybrid хандлагаар хийсэн апп нь гүйцэтгэлийн хувьд сул байдаг. Cross-platform хандлага нь нэг кодоос хоёр платформд хүрдэг тул цаг, зардлын хувьд хамгийн үр дүнтэй болохыг дараах харьцуулалт харуулна.

Хүснэгт 1.1: Мобайл аппликейшн хөгжүүлэлтийн арга хандлагуудын харьцуулалт

Шалгуур	**Native**	**Hybrid**	**Cross-platform**
Хэл/хэрэгсэл	Swift, Kotlin	HTML, JS	Flutter, React Native
Гүйцэтгэл	Хамгийн өндөр	Доогуур	Native-д ойр
Нэг кодын бааз	Үгүй	Тийм	Тийм
UI нийцтэй байдал	Ялгаатай	WebView хязгаарлагдмал	Бүрэн нийцтэй
Хөгжүүлэлтийн зардал	Өндөр (×2 баг)	Бага	Дунд
Энэ ажилд	—	—	**Flutter**

Эх: Зохиогчийн боловсруулалт

Flutter нь Google компанийн 2018 онд гаргасан нээлттэй эхийн UI фреймворк юм [6]. Flutter нь Dart хэл дээр бичигдсэн бөгөөд Skia/Impeller дүрслэлийн хөдөлгүүр ашиглан бүх UI-г платформаас үл хамааран өөрөө зурдаг. Энэ нь Android болон iOS-д яг ижил харагдах баталгаа олгодгийн зэрэгцээ нэмэлт platform-specific код бичих шаардлагыг арилгадаг. Хөгжүүлэлтийн явцад Hot Reload функц нь кодын өөрчлөлтийг апп-г дахин эхлүүлэхгүйгээр шууд харуулдаг тул хөгжүүлэлтийн хурдыг мэдэгдэхүйц нэмэгдүүлдэг. Flutter 3.x хувилбар нь Material Design 3 стандартыг бүрэн дэмждэг бөгөөд `ThemeData` классаар дамжуулан дизайн системийг нэгтгэн удирдах боломж олгодог [7]. Уг ажилд Flutter 3.x хувилбарыг ашигласан.

Dart нь Google-ийн боловсруулсан, объект хандлагат, хатуу төрлийн хэл юм [8]. Dart нь AOT (Ahead-of-Time) болон JIT (Just-in-Time) хоёуланг дэмждэг тул хөгжүүлэлтийн үед Hot Reload-ийн хурд, продакшн дээр AOT-ийн хурдан гүйцэтгэл

хоёуланг хангадаг. `async/await` синтакс нь Firebase Firestore-ийн болон API дуудлагуудыг уншихад хялбар байдлаар бичих боломж олгодог. Null Safety механизм нь null pointer алдааг compile цагт илрүүлдэг. Апп-ын дэлгэцийн төлвийг удирдахад StatefulWidget болон setState хослолыг ашиглав. Энэ нь нэмэлт dependency байхгүй, шинэ хөгжүүлэгчдэд ойлгомжтой, апп-ын хэмжээг жижиг байлгах давуу талтай.

Firebase нь Google компанийн санал болгодог BaaS (Backend as a Service) платформ бөгөөд мобайл апп хөгжүүлэгчдэд бэкэнд дэд бүтцийг өөрөө тохируулахгүйгээр бэлэн үйлчилгээнүүдийг ашиглах боломж олгодог [9]. Энэ ажилд Firebase-ийн гурван үндсэн үйлчилгээг ашигласан.

Cloud Firestore нь NoSQL, баримт жишгийн өгөгдлийн сан юм [10]. Бодит цагийн шинэчлэлт (real-time listeners), офлайн хадгалалт, автомат масштабчлал зэрэг онцлогтой бөгөөд collection-document бүтэц нь JSON форматтай ойролцоо тул Flutter SDK-аар шууд хандахад хялбар. Firestore-ийн security rules нь хэрэглэгч бүрийн хандах эрхийг нарийн тодорхойлдог тул аюулгүй байдлыг мобайл апп талаас хангаж чаддаг. Тухайн захиалагдсан цагийн хуваарийг бодит цагаар харуулах шаардлага байгаа тул real-time listener-ийг ашиглав.

Firebase Authentication нь хэрэглэгчийн нэвтрэлтийг удирдах бэлэн үйлчилгээ юм [11]. Email/нууц үгийн нэвтрэлт ашиглахад хэрэглэгч бүрт өвөрмөц UID үүсгэж, нэвтрэлтийн токен удирддаг. Энэ ажилд Node.js бэкэнд болон Nodemailer (Gmail SMTP)-ийн 6 оронтой баталгаажуулах кодтой хосолсон бөгөөд хэрэглэгчийн email хаягийн үнэн зөвийг баталгаажуулдаг.

Firebase Cloud Functions нь серверлэс тооцооллын үйлчилгээ бөгөөд AI чатботын үндсэн endpoint болгон ашигласан. Cloud Functions нь Anthropic-ийн API түлхүүрийг Flutter кодод задруулахгүйгээр аюулгүй хадгалах боломж олгодог. Хэрэглэгч чат илгээх бүрт Cloud Function дуудагдаж, Claude Haiku API-аас хариулт авч буцаана.

Захиалгын сануулгыг хэрэгжүүлэхэд **flutter_local_notifications** багцыг ашигласан [12]. Энэ багц нь Firebase Cloud Messaging (FCM) серверийн тусламжгүйгээр хэрэглэгчийн төхөөрөмжид мэдэгдлийг schedule хийх боломжтой бөгөөд интернет холболтгүй тохиолдолд ч ажилладаг. Захиалга хийх бүрт тухайн захиалгаас 1 цагийн өмнөх цагт сануулга schedule хийгддэг.

1.2 Бэкэнд хөгжүүлэлтийн технологи

Мобайл апп дан ганцаар бүх логикийг агуулбал API түлхүүр болон нэвтрэлтийн нууцлал клиент кодод задарч аюулгүй байдал эмзэг болно. Мөн email илгээх, AI чатбот, захиалгын давхардал шалгах зэрэг серверийн талын логикийг Flutter-т хийх боломжгүй. Иймд Node.js/Express-д суурилсан тусдаа бэкэнд сервер шаардлагатай болсон. Бэкэнд нь Flutter апп болон Firebase хоорондын зуучлагчийн үүрэг гүйцэтгэж, email баталгаажуулалт, AI чат, захиалгын менежмент зэрэг функцийг нэгтгэн хангадаг.

Node.js нь Chrome V8 JavaScript хөдөлгүүр дээр суурилсан серверийн тал JavaScript ажиллуулах орчин юм [13]. Node.js-ийн event-driven, non-blocking I/O загвар нь олон нэгэн зэрэг холболтыг цөөн нөөцөөр боловсруулах чадвартай. npm экосистем нь Nodemailer (email), node-cron (scheduled jobs), Groq SDK зэрэг шаардлагатай бүх багцыг хурдан суулгах боломж олгодог. **Express.js** нь Node.js дээр суурилсан хөнгөн, уян хатан веб фреймворк юм [14]. Middleware систем нь хүсэлт боловсруулах дамжлагыг зохион байгуулах боломж олгодог бөгөөд Router нь URL

чиглүүлэлтийг модуль болгон хуваана. Node.js болон Express хосолсноор JSON боловсруулалт, REST API хурдан бариход маш тохиромжтой болдог.

REST API (Representational State Transfer) нь Roy Fielding (2000) [15] боловсруулсан веб үйлчилгээний архитектурын хэв маяг юм. HTTP протоколын GET, POST, PUT, DELETE аргуудыг ашиглан нөөцийг удирддаг. Сервер хүсэлт бүрийг бие даасан гэж үздэг (stateless) бөгөөд нөөц бүрийг URL-ээр илэрхийлнэ. Энэ ажилд Flutter апп болон Node.js бэкэнд хоорондын бүх харилцааг RESTful API-аар, JSON форматаар гүйцэтгэдэг. Системд нийт 8 API endpoint байдаг бөгөөд тэдгээрийг доорх хүснэгтэд нэгтгэн харуулав.

Хүснэгт 1.2: Node.js/Express REST API endpoint-уудын жагсаалт

Арга	**Endpoint**	**Тайлбар**
POST	`/api/auth/send-code`	6 оронтой баталгаажуулах код email-р илгээх
POST	`/api/auth/verify-code`	Оруулсан кодыг шалгаж, email баталгаажуулах
POST	`/api/users/create`	Шинэ хэрэглэгч Firestore-д бүртгэх
GET	`/api/bookings/:venueId`	Заалны захиалгуудын жагсаалт авах
POST	`/api/bookings/create`	Шинэ захиалга үүсгэх
PUT	`/api/bookings/:id/cancel`	Захиалга цуцлах
GET	`/api/schedule/:venueId`	Цагийн хуваарь болон тогтмол захиалга авах
POST	`/api/chat`	Groq Llama AI чатбот хариулт авах (нөөц горим)

Эх: Зохиогчийн боловсруулалт

Docker нь аппликейшн болон түүний бүх хамаарлыг тусгаарлагдсан контейнерт багцалдаг платформ юм [16]. Docker ашиглахын гол давуу тал нь хөгжүүлэлтийн болон продакшн орчин яг ижил байх баталгаа юм: хөгжүүлэгчийн компьютерт ажиллаж байгаа бэкэнд яг л AWS EC2-д ажилладагтай ижил байдаг. Dockerfile нь Node.js 18 Alpine суурь дүрс дээр package.json суулгаж, эх кодыг хуулан, 3000 портоор сервер эхлүүлдэг тохиргооны файл юм. Alpine суурь дүрс сонгосноор эцсийн дүрсний хэмжээ жижиг байж, татах, байршуулах хугацаа хурдан болдог. Docker контейнер хэдхэн секундэд эхлэх бөгөөд нэмэлт тохиргоо шаардаггүй тул EC2-д байршуулалтыг хялбарчилдаг.

1.3 Үүлэн орчин ба автоматжуулалт

Тусдаа бэкэнд сервер хэрэгтэй болсон тул тэрийгээ ямар дэд бүтцэд байршуулах вэ гэсэн шийдвэр гаргах шаардлагатай болсон. Локал сервер ашиглах нь тасралтгүй ажиллагаа болон дэлхийн аль ч газраас хандах боломжийг хязгаарладаг. AWS (Amazon Web Services) үүлэн үйлчилгээ нь дэлхийн дэд бүтцэд суурилсан, найдвартай uptime баталгаатай, Docker контейнер ажиллуулах бүрэн дэмжлэгтэй шийдэл тул энэ ажилд сонгосон.

Amazon EC2 (Elastic Compute Cloud) нь AWS-ийн виртуал сервер үйлчилгээ бөгөөд Node.js/Express бэкэнд ap-southeast-1 (Сингапур) бүсийн EC2 t3.micro instance дээр ажиллана [17]. Сингапур бүсийг Монголоос харьцангуй ойр байгаа AWS бүс тул сонгосон бөгөөд энэ нь API-ийн хариу цагийг бууруулна. EC2 нь серверийн тохиргоог бүрэн хянах, security group-ийн тусламжтай сүлжээний хандалтыг нарийн удирдах боломж олгодог.

Amazon ECR (Elastic Container Registry) нь AWS-ийн Docker дүрсний хадгалалтын үйлчилгээ юм [18]. GitHub Actions-аар build хийсэн Docker дүрсийг ECR-д push хийж, EC2 instance тэр дүрсийг татан ажиллуулдаг. ECR нь AWS IAM эрхийн удирдлагатай нэгдсэн тул дүрсний нууцлал баталгаатай, EC2-тэй нэгдмэл ажилладаг.

GitHub Actions нь GitHub платформд суурилагдсан CI/CD (Continuous Integration / Continuous Deployment) автоматжуулалтын хэрэгсэл юм [19]. YAML файлаар тодорхойлсон workflow нь тодорхой нөхцөл биелэхэд автоматаар ажилладаг. Энэ ажилд `backend/` хавтасны файл өөрчлөгдөх бүрт CI/CD хоолой автоматаар ажиллаж, Docker image build хийж AWS ECR-д push хийж, EC2-д шинэ хувилбарыг 3–4 минутын дотор байршуулдаг. Хөгжүүлэгч зөвхөн `git push` хийхэд л бэкэнд автоматаар шинэчлэгддэг тул хүний гараар сервер дээр нэвтрэх шаардлага байхгүй болдог. GitHub Secrets нь AWS, Groq, Anthropic-ийн API нууцлал болон EC2-ийн SSH түлхүүрийг аюулгүй хадгалдаг.

AI чатботын хувьд хоёр загвар хосолсон нөөцлөлтийн архитектурыг ашигласан. **Anthropic Claude Haiku 4.5** нь Firebase Cloud Functions-д суурилсан үндсэн горимын загвар юм [20]. Хурдан, хэмнэлттэй хариулт өгдөг бөгөөд API түлхүүр Flutter кодод задрахгүй Cloud Functions-д аюулгүй хадгалагддаг. **Groq Llama 3.1-8b** нь Node.js бэкэнд дамжуулан ажиллах нөөц горимын загвар юм [21]. Groq-ийн LPU (Language Processing Unit) дэд бүтэц нь маш хурдан хариулт өгдөг давуу талтай. Хэрэв Claude Haiku удаан буюу алдаа гарвал Flutter апп автоматаар Groq руу шилжих тул AI чатботын найдвартай байдал нэмэгдэнэ.

Ашигласан технологиудын бүрэн жагсаалт, тус бүрийн үүрэг болон сонгосон шалтгааныг доорх хүснэгтэд нэгтгэн харуулав. Энэхүү технологийн стек нь системийн шаардлагад нийцсэн, найдвартай, масштабчлах боломжтой шийдлийг бүрдүүлнэ.

Хүснэгт 1.3: Ашигласан технологийн стекийн нэгтгэсэн хүснэгт

Технологи	**Үүрэг**	**Сонгосон шалтгаан**
Flutter 3.x	Мобайл клиент (Android/iOS)	Нэг кодоос хоёр платформ, Material Design 3
Dart	Flutter хэл	AOT compile, Null Safety, async/await
Firebase Auth	Нэвтрэлтийн систем	Аюулгүй, Google-ийн үүлэн дэмжлэгтэй
Cloud Firestore	NoSQL өгөгдлийн сан	Real-time listener, офлайн дэмжлэг
Cloud Functions	Serverless AI endpoint	API түлхүүрийг аюулгүй хадгалах
Node.js/Express	REST API бэкэнд	Event-driven, JSON API бариход тохиромжтой
Docker	Контейнержуулалт	Орчны тогтвортой байдал
AWS EC2/ECR	Үүлэн сервер ба registry	Бүрэн хяналт, Docker дэмжлэг
GitHub Actions	CI/CD хоолой	Git workflow-той нэгдмэл, автомат байршуулалт
Claude Haiku 4.5	AI чатбот (үндсэн)	Хурдан, хэмнэлттэй LLM
Groq Llama 3.1-8b	AI чатбот (нөөц)	LPU дэд бүтэц, маш хурдан хариулт
flutter_local_notifications	Локал мэдэгдэл	FCM сервергүйгээр мэдэгдэл

Эх: Зохиогчийн боловсруулалт

БҮЛЭГ 2

СИСТЕМИЙН ЗАГВАРЧЛАЛ БА ХӨГЖҮҮЛЭЛТ

2.1 Системийн шаардлага ба загварчлал

2.1.1 Функциональ болон функциональ бус шаардлага

Системийн шаардлагыг Удиртгалд тогтоосон таван асуудалд тулгуурлан боловсруулав: утсаар захиалах хүндрэл, биечлэн очих зайлшгүй байдал, цагийн мэдээлэл ил тод бус байдал, захиалга цуцлах боломжгүй байдал, сануулга байхгүй байдал. Эдгээр асуудал бүрийг шийдвэрлэх функциональ шаардлагыг дараах байдлаар тодорхойлсон.

Хэрэглэгчийн функцийн хувьд: email хаяг болон нууц үгээр бүртгүүлэх, бүртгэлийн үед тухайн email рүү 6 оронтой баталгаажуулах код илгээгдэж, кодоо оруулсны дараа л нэвтрэх боломж нээгдэх; таван спорт байгууламжийн жагсаалтыг нүүр дэлгэцэд харах; тухайн заалны боломжтой болон хаагдсан цагийн хуваарийг харах; цаг захиалах, DBK-xxxxx форматын захиалгын код авах; идэвхтэй захиалгаа цуцлах; захиалгын түүхийг харах; захиалгаас 1 цагийн өмнө локал мэдэгдэл хүлээн авах; AI чатботтой спорт заалтай холбоотой асуулт асуух. Системийн автомат функцийн хувьд 15 минут тутамд cron ажил дуусгавар болсон захиалгыг `completed` төлөвт шилжүүлэх, байгууллагуудын тогтмол (давтагдах) захиалгыг `fixed_bookings` цуглуулгаас уншиж долоо хоног бүр цагийн хуваарьт тусгах зэрэг функц орно.

Функциональ бус шаардлагыг хэмжигдэхүйц хэлбэрээр тодорхойлсон бөгөөд тэдгээрийг доорх хүснэгтэд нэгтгэв.

Хүснэгт 2.1: Функциональ бус шаардлагуудын хүснэгт

Ангилал	**Шаардлага**	**Хэмжих арга**
Гүйцэтгэл	Апп 3 секундын дотор ачаалах; API хариулт 2 секундын дотор	Хэрэглэгчийн цагийн хэмжилт
Найдвартай байдал	Firebase 99.9%, AWS EC2 99.5% uptime	AWS CloudWatch
Аюулгүй байдал	Нэвтрэлт Firebase Auth-аар; API нууцлал GitHub Secrets + .env	OWASP Mobile Security
Хэрэглэхэд хялбар	Material Design 3 стандарт; 5/5-аас 4.0+ оноо	Usability Scale
Дэмжих платформ	Android 8.0+ болон iOS 13.0+	Бодит төхөөрөмж туршилт

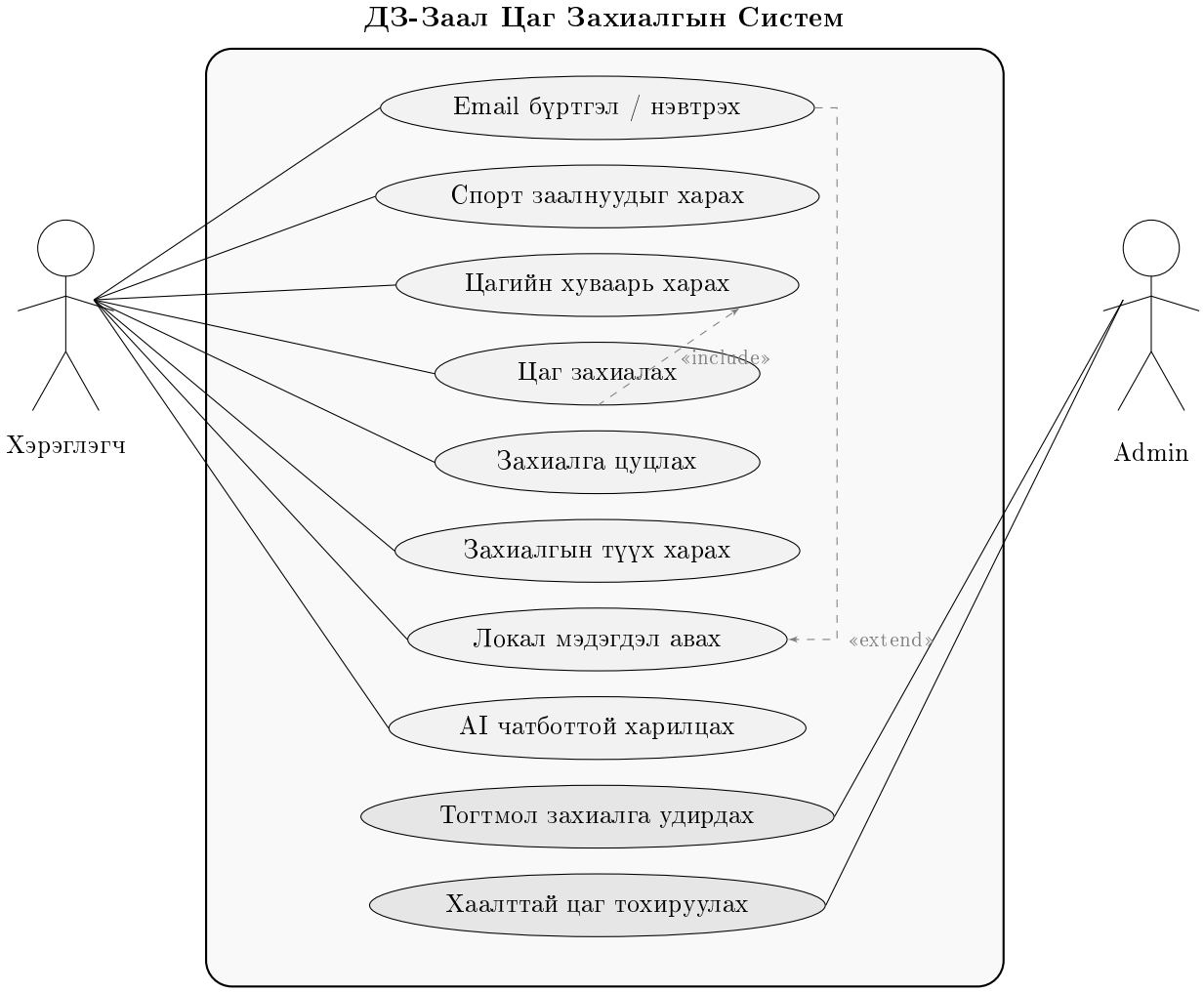
Эх: Зохиогчийн боловсруулалт

2.1.2 UML Диаграммууд

Use Case Diagram нь системийн гадаад оролцогч болон тэдгээрийн гүйцэтгэх үндсэн функцуудыг тодорхойлно. Систэмд хэрэглэгч (иргэн) болон Admin (заалны цаг олгогч) гэсэн хоёр actor оролцоно.

Цаг захиалах үйл явцын дараалыг Activity Diagram-аар дүрсэлсэн. Хэрэглэгч нэвтэрсний дараа заал, огноо, цагийн үе сонгож захиалга үүсгэнэ. Сонгосон цаг аль хэдийн захиалагдсан бол систем алдаа мэдэгдэж өөр цаг сонгохыг хүснэ.

Захиалга үүсгэх үйл явцад оролцогч тал бүрийн харилцан үйлдлийн дараалыг Sequence Diagram-аар илэрхийлсэн. Хэрэглэгч цагийг сонгоход Flutter апп Firestore-аас хаагдсан цагийн жагсаалтыг авч, захиалга батлагдсаны дараа Firestore-д бичиж DBK код буцаана. Локал мэдэгдлийн schedule-г апп өөрөө хийдэг.



Зураг 2.1: Use Case Diagram — хэрэглэгч болон Admin-ийн харилцаа
Эх: Зохиогчийн боловсруулалт

Email баталгаажуулалтын дараалал нь Firebase Auth, Node.js бэкэнд (Nodemailer) болон Firestore гурвыг хамардаг тул тусад нь дараах диаграммаар дүрсэлсэн.

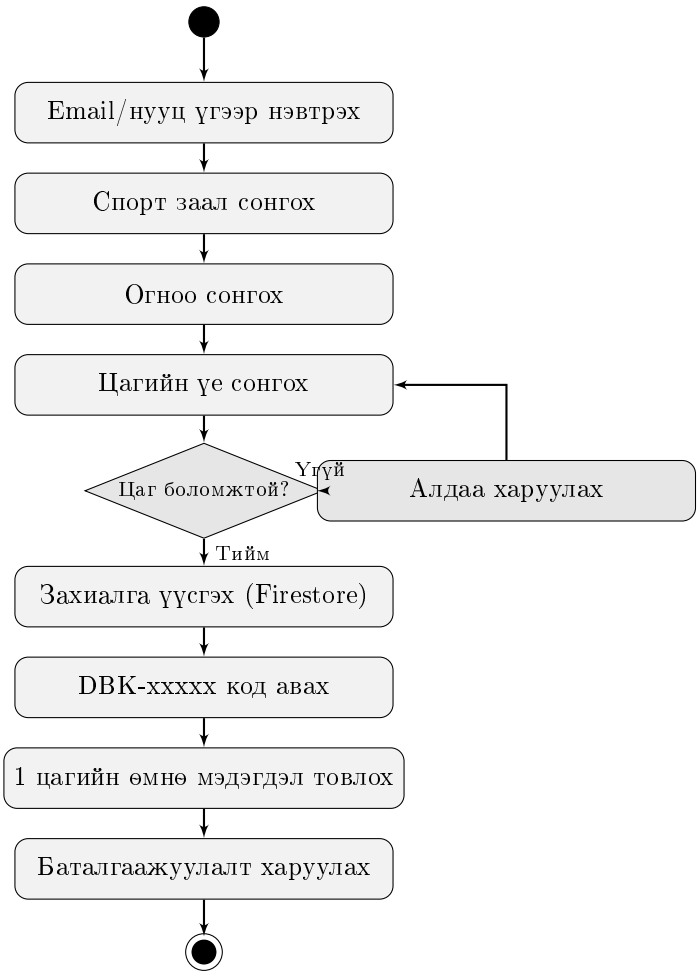
2.1.3 Системийн архитектур

Систем нь гурван давхар архитектурт тулгуурлана: Flutter мобайл клиент, Node.js/Express бэкэнд (AWS EC2), Firebase үйлчилгээнүүд. Flutter апп нь `lib/config.dart`-д тодорхойлсон URL-аар AWS EC2 бэкэндтэй REST API дамжуулан, Firebase SDK-р Cloud Firestore болон Authentication-тай шууд холбогддог. AI чатботын хоёр горим (Claude Haiku болон Groq Llama) нь тус тусын үүлэн API-тай холбогдоно.

2.2 Frontend хөгжүүлэлт

2.2.1 Дизайн систем ба навигаци

Апп-ын дизайн нь говийн цөлийн дулаан тонийг илэрхийлсэн, нүдэнд ээлтэй цайвар дэвсгэр дээрх наранжин-хүрэн өнгөн схемтэй. Бежийн дэвсгэр (`#EDE7DA`), наранжин (`#D4700F`), гүн наранжин (`#B85C08`), харанхуй хүрэн (`#1A1208`) дөрвөн үндсэн

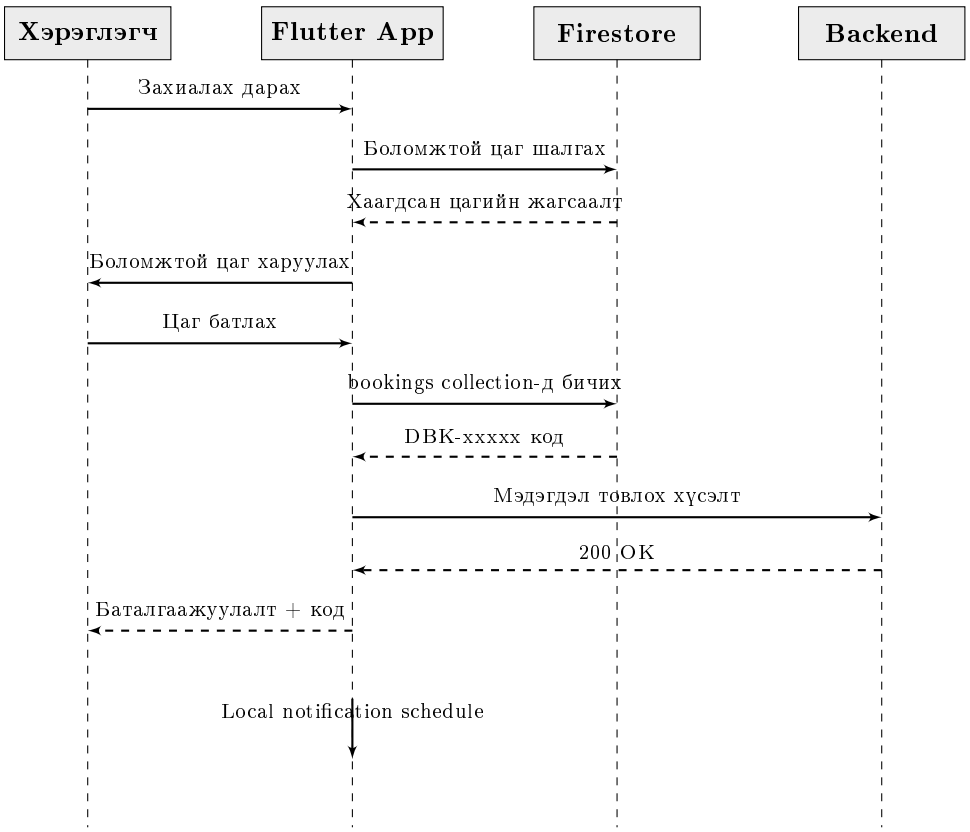


Зураг 2.2: Цаг захиалах үйл ажиллагааны диаграмм (Activity Diagram)
Эх: Зохиогчийн боловсруулалт

өнгийг `Montserrat` фонттой хослуулав. Material Design 3-ийн `ThemeData` классаар дамжуулан апп-ын бүх widget-д нэгдсэн дизайн систем хэрэгжүүлсэн:

Эх код 2.1: Material Design 3 дулаан говийн дизайн тохиргоо — AppTheme класс

```
class AppTheme {
  static const Color primary    = Color(0xFFEDE7DA);
  static const Color secondary  = Color(0xFFD4700F);
  static const Color accent     = Color(0xFFB85C08);
  static const Color textPrimary = Color(0xFF1A1208);

  static ThemeData get lightTheme => ThemeData(
    useMaterial3: true,
    fontFamily: 'Montserrat',
    brightness: Brightness.light,
    scaffoldBackgroundColor: primary,
    colorScheme: const ColorScheme.light(
      primary:  secondary,
      secondary: accent,
```



Зураг 2.3: Захиалга үүсгэх дараалын диаграмм (Sequence Diagram)

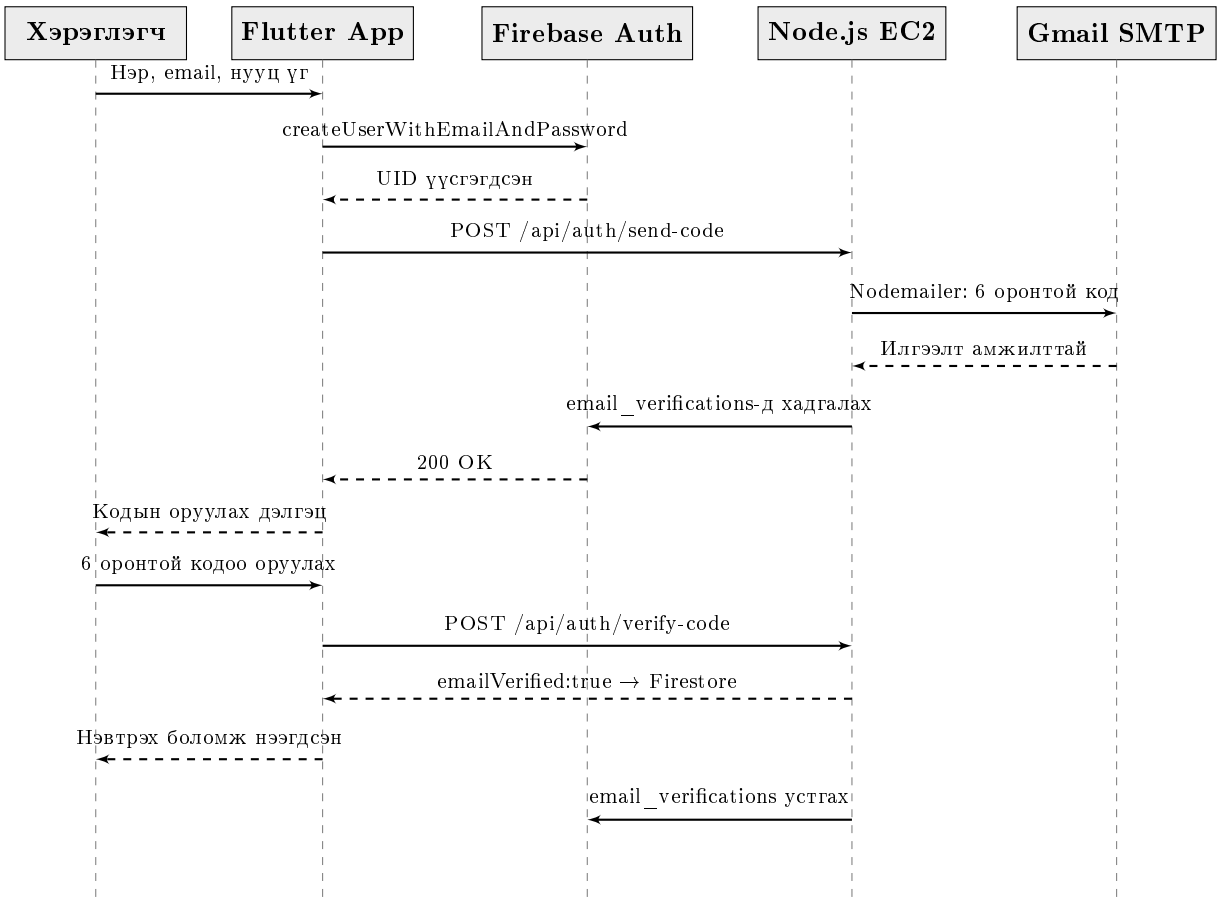
Эх: Зохиогчийн боловсруулалт

```
    ),
  );
}
```

Апп-ын үндсэн навигаци нь `BottomNavigationBar` ашиглан 4 tab-тай: Нүүр (Home), Захиалгууд (Bookings), Чат (Chat), Профайл (Profile). Идэвхтэй tab-ийн индексийг root `StatefulWidget`-ийн `setState`-д хадгалдаг. Навигацийн бүтэц болон дэлгэцүүдийн удирдлагыг доорх кодоор хэрэгжүүлсэн:

Эх код 2.2: BottomNavigationBar — 4 tab навигаци, MainScreen класс

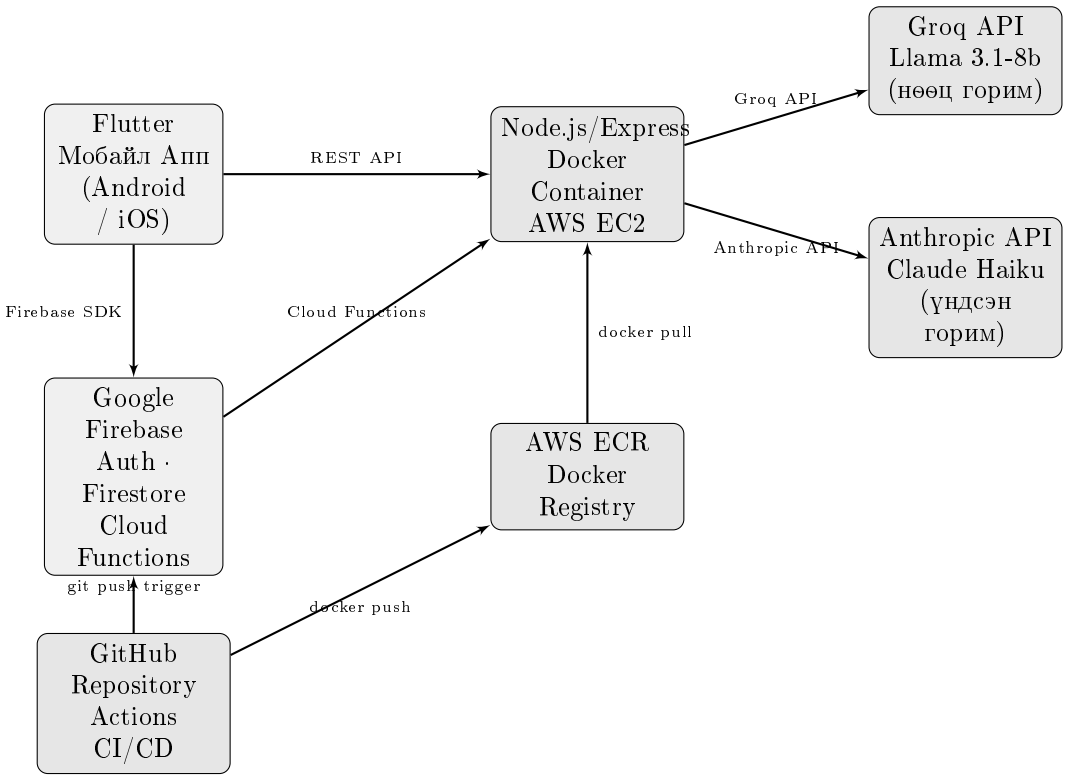
```
class _MainScreenState extends State<MainScreen> {
  int _selectedIndex = 0;
  final List<Widget> _screens = const [
    HomeScreen(), BookingsScreen(),
    ChatScreen(), ProfileScreen(),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: _screens[_selectedIndex],
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _selectedIndex,
```



Зураг 2.4: Email баталгаажуулалтын дараалын диаграмм
Эх: Зохиогчийн боловсруулалт

```
      onTap: (index) => setState(() => _selectedIndex = index),
      items: const [
        BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
        BottomNavigationBarItem(icon: Icon(Icons.calendar_today), label
            : 'Bookings'),
        BottomNavigationBarItem(icon: Icon(Icons.chat), label: 'Chat'),
        BottomNavigationBarItem(icon: Icon(Icons.person), label: '
            Profile'),
      ],
    ),
  );
 }
}
```

Flutter апп нь давхаргат бүтэцтэй: `lib/screens/` дэлгэцүүд, `lib/widgets/` дахин ашиглагдах widget-ууд, `lib/services/` Firebase болон API холболтын классууд, `lib/models/` өгөгдлийн загварууд, `lib/config.dart` тохиргооны нэг эх сурвалж. Апп-ын бүтэн архитектурыг доорх диаграммаар харуулав.



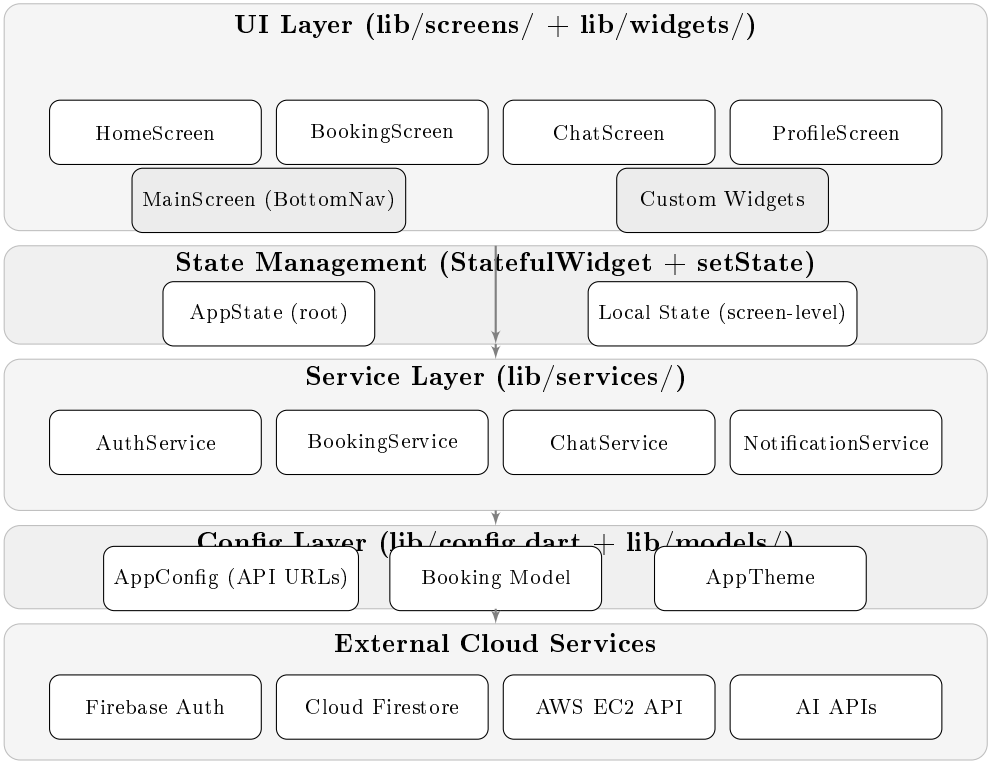
Зураг 2.5: Системийн гурван давхаргат үүлэн архитектур
Эх: Зохиогчийн боловсруулалт

2.2.2 Нэвтрэлт болон цаг захиалгын хэрэгжүүлэлт

Нэвтрэлтийн дэлгэцэд хэрэглэгч email хаяг болон нууц үгээр бүртгүүлж нэвтрэх боломжтой. Анхны бүртгэлийн үед тухайн email хаягт 6 оронтой баталгаажуулах код илгээгдэх бөгөөд кодоо зөв оруулсны дараа л нэвтрэх боломж нээгдэнэ. Бүртгэлийн урсгал: хэрэглэгч нэр, email, нууц үгээ оруулна; Firebase Auth-д `createUserWithEmailAndPassword` дуудаж UID үүсгэнэ; Backend-ийн `POST /api/auth/send-code` endpoint дуудаж 6 оронтой кодыг email рүү илгээнэ; хэрэв код илгээхэд алдаа гарвал Firebase Auth хэрэглэгчийг автоматаар устгаж (rollback) дахин оролдох боломж нээгдэнэ; хэрэглэгч кодоо оруулахад `POST /api/auth/verify-code` дуудаж баталгаажуулна; амжилттай бол `emailVerified: true` тэмдэглэгдэж апп нүүр дэлгэцэд шилжинэ.

Эх код 2.3: Firebase Email Authentication — бүртгэлийн Flutter код

```
Future<void> registerWithEmail(
   String email, String password, String name) async {
  final credential = await FirebaseAuth.instance
      .createUserWithEmailAndPassword(
        email: email, password: password);
  final uid = credential.user!.uid;

  await http.post(
    Uri.parse(AppConfig.authEndpoint + '/send-code'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'email': email, 'uid': uid}),
  );
```



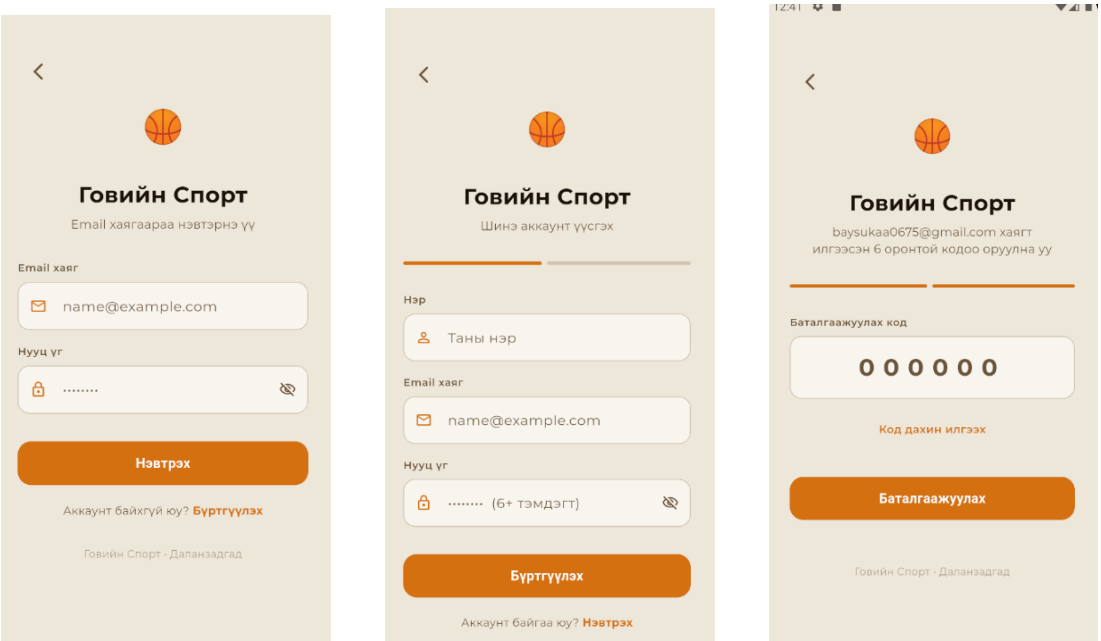
Зураг 2.6: Flutter аппликейшний давхаргат архитектур
Эх: Зохиогчийн боловсруулалт

```
}
```

Нүүр дэлгэцэд таван спорт байгууламжийн карт харагдана. Заалнуудын мэдээлэл кодод шууд тодорхойлсон (hardcoded) тул Firestore хандалт шаардлагагүй бөгөөд дэлгэц нэн даруй ачаалдаг. Хэрэглэгч карт дарахад тухайн заалны цагийн хуваарийн дэлгэц нээгддэг. Цагийн хуваарийн дэлгэцэд хэрэглэгч огноо сонгоход тухайн өдрийн 16 цагийн үе (08:00–24:00, нэг цагийн алхамтай) харагдана. Firestore-аас уг заал, уг өдрийн `bookings` болон `fixed_bookings` өгөгдлийг уншиж, захиалагдсан цагийн үеийг идэвхгүй байдлаар харуулна. Захиалга хийхдээ `DBK-xxxxx` форматын захиалгын код автоматаар үүснэ.

Эх код 2.4: Цагийн хуваарь болон захиалгын код үүсгэх логик

```
Future<void> _loadSlots(String venueId, String date) async {
  final bookSnap = await FirebaseFirestore.instance
      .collection('bookings')
      .where('venueId', isEqualTo: venueId)
      .where('date', isEqualTo: date)
      .where('status', whereIn: ['upcoming', 'active'])
      .get();

  final weekday = DateTime.parse(date).weekday % 7;
  final fixedSnap = await FirebaseFirestore.instance
      .collection('fixed_bookings')
      .where('venueId', isEqualTo: venueId)
      .where('dayOfWeek', isEqualTo: weekday)
      .where('isActive', isEqualTo: true)
```



(A) Нэвтрэх дэлгэц (B) Бүртгүүлэх дэлгэц (C) Email баталгаажуулах

Зураг 2.7: Email/нууц үгийн нэвтрэлт болон баталгаажуулалтын дэлгэцүүд
Эх: Зохиогчийн зураг авалт

```
15      .get();
16
17  final bookedSlots = {
18    ...bookSnap.docs.map((d) => d['timeSlot'] as String),
19    ...fixedSnap.docs.map((d) => d['timeSlot'] as String),
20  };
21  setState(() => _bookedSlots = bookedSlots);
22 }
```

`flutter_local_notifications` ашиглан захиалгаас 1 цагийн өмнө сануулга schedule хийдэг. Захиалга хийх бүрт тухайн захиалгаас 1 цагийн өмнөх цагт мэдэгдэл schedule хийгдэж, FCM сервер байхгүй тул бүрэн офлайн ажилладаг. Android болон iOS аль алинд нэг кодоор хэрэгжинэ.

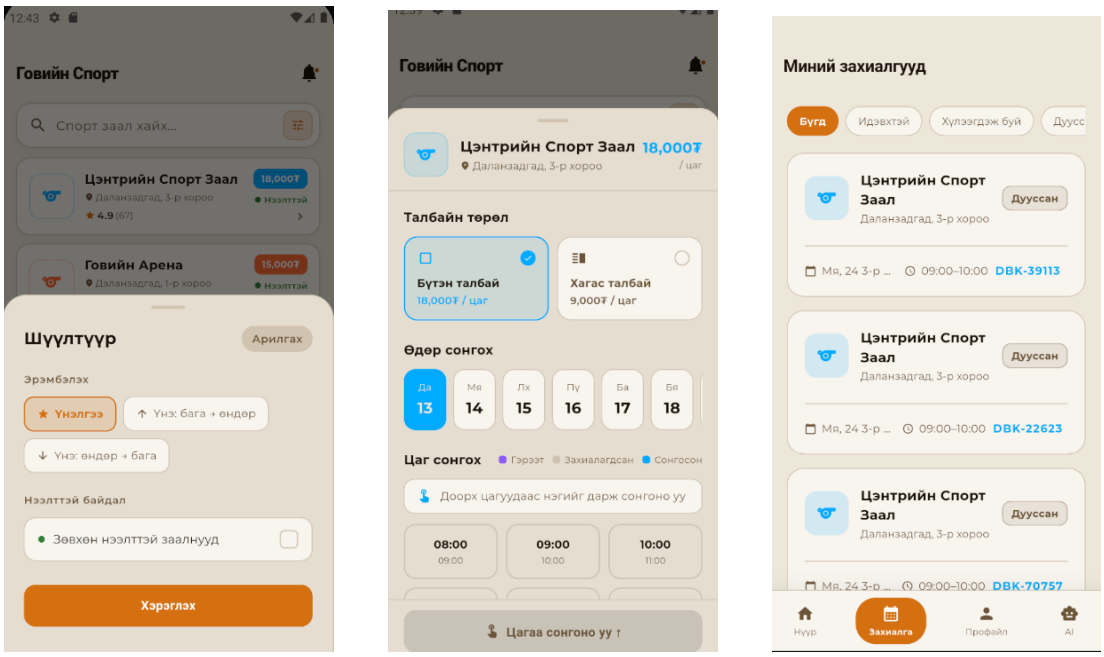
2.3 Backend хөгжүүлэлт

2.3.1 Express.js API ба email баталгаажуулалт

Node.js/Express бэкэнд нь `backend/` хавтаст байрлах бөгөөд `server.js` үндсэн файл, `routes/auth.js` email баталгаажуулалт, `routes/bookings.js` захиалгын удирдлага, `routes/chat.js` AI чат, `routes/schedule.js` цагийн хуваарь гэсэн route модулиудаас бүрдэнэ. Апп-ын бүх API хаягийг `lib/config.dart` файлд төвлөрүүлснээр хөгжүүлэлтийн болон продакшн орчинд шилжихэд нэг файлыг л өөрчлөх хэрэгтэй болдог.

Эх код 2.5: Express.js — үндсэн route тохиргоо (server.js)

```
1 const express = require('express');
2 const app     = express();
```



(A) Нүүр — заалны жагсаалт (B) Цаг захиалах дэлгэц (C) Миний захиалгууд

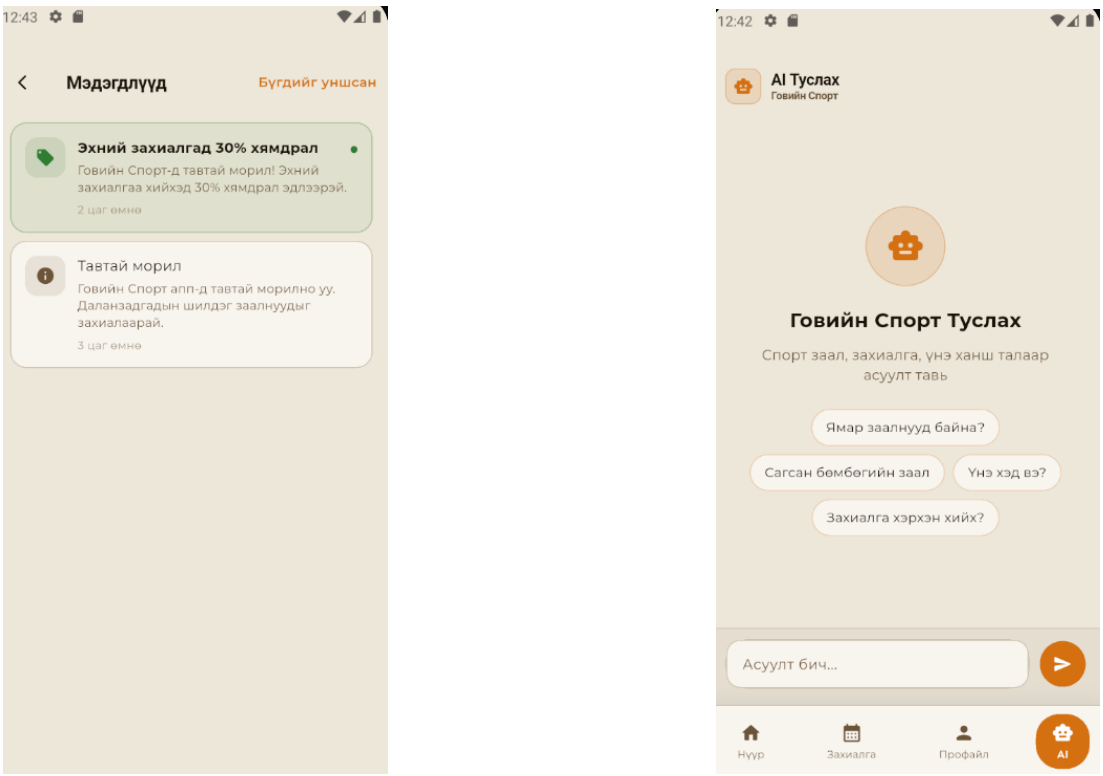
Зураг 2.8: Нүүр дэлгэц, цаг захиалах болон захиалгын жагсаалтын дэлгэцүүд
Эх: Зохиогчийн зураг авалт

```
app.use(express.json());
app.use('/api/auth',   require('./routes/auth'));
app.use('/api/chat',   require('./routes/chat'));
app.use('/api/users',  require('./routes/users'));
app.use('/api/bookings', require('./routes/bookings'));
app.use('/api/schedule', require('./routes/schedule'));
app.listen(3000, () => console.log('Server running on port 3000'));
```

Email баталгаажуулалтын endpoint нь 6 оронтой санамсаргүй код генерлэж, Firestore-д 10 минутын хугацаатайгаар хадгалж, Nodemailer-ээр Gmail SMTP дамжуулан тухайн email рүү илгээдэг. Хугацаа хэтэрсэн код дахин илгээх шаардлагатай бөгөөд код шалгасны дараа `email_verifications` document автоматаар устгагддаг.

Эх код 2.6: Backend — Nodemailer ашиглан баталгаажуулах код илгээх

```
router.post('/send-code', async (req, res) => {
  const { email, uid } = req.body;
  const code = Math.floor(100000 + Math.random() * 900000).toString();

  await db.collection('email_verifications').doc(uid).set({
    code, email,
    expiresAt: new Date(Date.now() + 10 * 60 * 1000),
  });

  const transporter = nodemailer.createTransport({
    host: 'smtp.gmail.com', port: 587,
    auth: { user: process.env.EMAIL_USER,
```



Зураг 2.9: Захиалгын сануулга болон AI туслахын дэлгэцүүд
Эх: Зохиогчийн зураг авалт

```
13            pass: process.env.EMAIL_PASS },
14  });
15  await transporter.sendMail({
16    from: `"Goviin Sport" <${process.env.EMAIL_USER}>`,
17    to: email, subject: 'Goviin Sport - Batalgaajuulakh kod',
18    html: `<h2>${code}</h2><p>10 minutiin dotoor huchintei</p>`,
19  });
20  res.json({ success: true });
21 });
```

2.3.2 AI чатбот болон cron ажил

AI чатботын архитектур нь хоёр давхар горимтой: Firebase Cloud Functions дамжуулан Anthropic Claude Haiku 4.5 (үндсэн горим), Node.js бэкэнд дамжуулан Groq Llama 3.1-8b (нөөц горим). Cloud Functions дотор Anthropic-ийн SDK ашиглан Claude Haiku API-тай харилцана. Хэрэглэгч нэвтэрсэн тохиолдолд л чат endpoint дуудагдах бөгөөд API түлхүүр Flutter кодод задрахгүй хамгаалагддаг.

Эх код 2.7: Firebase Cloud Functions — Claude Haiku AI чат endpoint

```
1 exports.chat = functions.https.onCall(async (data, context) => {
2   if (!context.auth) throw new functions.https.HttpsError(
3       'unauthenticated', 'Authentication required');
4   const client = new Anthropic({ apiKey: functions.config().anthropic.key
        });
5   const message = await client.messages.create({
```

```
    model: 'claude-haiku-4-5', max_tokens: 512,
    messages: [{ role: 'user', content: data.message }],
    system: 'You are a sports hall booking assistant for Dalanzadgad city
        .',
  });
  return { reply: message.content[0].text };
});
```

Groq нөөц горимын endpoint нь `/api/chat` хаягаар POST хүсэлт хүлээн авч Groq SDK-аар Llama 3.1-8b загвараар хариулт авдаг. Claude алдаа гарсан тохиолдолд Flutter апп автоматаар энэ endpoint рүү хандана.

Бэкэнд нь 15 минут тутамд cron ажлыг ажиллуулж, эхлэх цаг нь өнгөрсөн боловч `upcoming` төлөвтэй захиалгуудыг `completed` болгон шинэчилдэг. Ингэснээр Firestore-д хэрэглэгчийн захиалгын мэдээлэл бодит байдалтай нийцсэн байдлаар шинэчлэгдэнэ.

Эх код 2.8: Cron ажил — 15 минут тутамд дуусан захиалга шинэчлэх

```
cron.schedule('*/15 * * * *', async () => {
  const now  = new Date();
  const today = now.toISOString().split('T')[0];
  const nowTime = now.getHours().toString().padStart(2,'0') + ':'
              + now.getMinutes().toString().padStart(2,'0');
  const snapshot = await admin.firestore()
    .collection('bookings')
    .where('status', '==', 'upcoming')
    .where('dateKey', '<=', today).get();
  const batch = admin.firestore().batch();
  snapshot.docs.forEach(doc => {
    const endTime = doc.data().timeSlotEnd;
    if (doc.data().dateKey < today ||
        (doc.data().dateKey === today && endTime <= nowTime)) {
      batch.update(doc.ref, { status: 'completed' });
    }
  });
  await batch.commit();
});
```

2.4 Өгөгдлийн сан

Cloud Firestore-д дөрвөн collection ашигласан. Спорт заалнуудын мэдээлэл Firestore-д хадгалагддаггүй — кодод шууд тодорхойлсон байдлаар тавьж, Firestore-ийн унших тоог хэмнэдэг.

users collection нь хэрэглэгч бүрийн мэдээллийг хадгалдаг бөгөөд document ID нь Firebase Auth-ийн UID байна. uid, name, email, emailVerified (boolean), createdAt (timestamp) талбаруудтай. Хэрэглэгч бүртгүүлэх үед `emailVerified: false`, баталгаажсаны дараа `emailVerified: true` болдог.

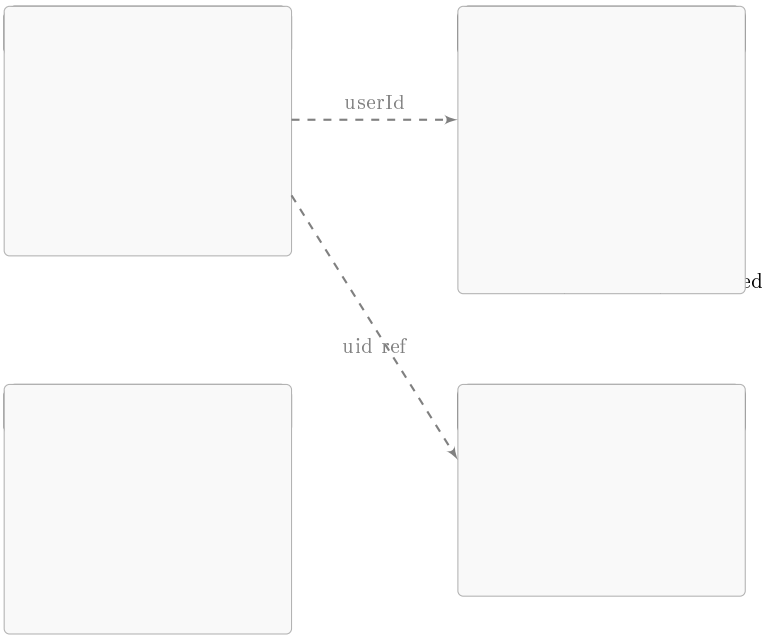
bookings collection нь хэрэглэгчдийн бүх захиалгыг хадгалдаг. DBK-xxxxx форматын захиалгын код (code), хэрэглэгчийн UID (userId), email (userEmail), нэр (userName), заалны ID (venueId), огноо (date, dateKey), цагийн эхлэл/төгсгөл (timeSlot, timeSlotEnd), талбайн төрөл (courtType: full|half), үнэ (price), төлөв (status: upcoming|cancelled|com

талбаруудтай. dateKey нь YYYY-MM-DD форматаар хадгалагддаг бөгөөд Firestore-ийн шүүлтийн хурдыг нэмэгдүүлнэ.

fixed_bookings collection нь байгууллагуудын долоо хоногт давтагдах тогтмол цагийг хадгалдаг. venueId (заалны ID), organizationName (байгууллагын нэр), dayOfWeek (долоо хоногийн өдөр: 0=Ням—6=Бямба), timeSlot (цагийн үе), isActive (идэвхтэй эсэх) талбаруудтай. Энэ collection-оос уншсан өгөгдлөөр засгийн газар, компаниудын тогтмол цагийг цагийн хуваарьт тусгаж, хэрэглэгч тэр цагийг захиалах боломжгүй болгодог.

email_verifications collection нь бүртгэлийн баталгаажуулах кодыг түр хадгалдаг. Document ID нь Firebase Auth-ийн UID байна. code (6 оронтой тоон код), email, expiresAt (10 минутын дараа дуусах), createdAt талбаруудтай. Баталгаажуулалт амжилттай болсны дараа тухайн document автоматаар устгагддаг.

Collection хоорондын холбоосыг дараах диаграммаар харуулав.



Зураг 2.10: Firestore collection-уудын схем диаграмм
Эх: Зохиогчийн боловсруулалт

БҮЛЭГ 3

ХЭРЭГЖҮҮЛЭЛТ

3.1 Үүлэн байршуулалт

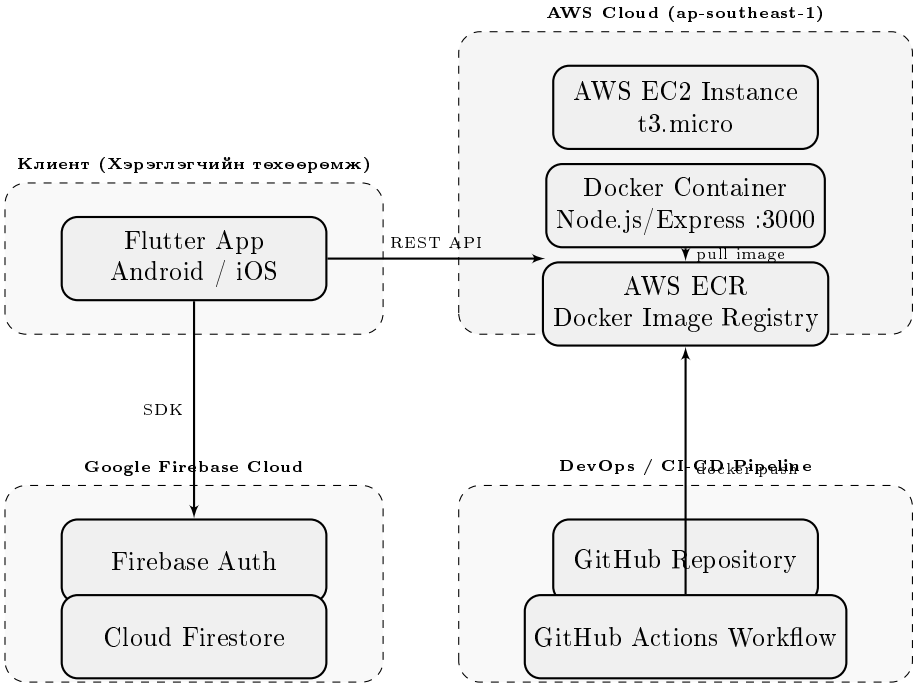
3.1.1 Docker контейнержуулалт ба AWS EC2

Node.js/Express бэкэнд сервер нь Docker контейнер хэлбэрээр AWS EC2 (ap-southeast-1, Сингапур) дээр ажиллана. Docker ашиглахын гол зорилго нь хөгжүүлэлтийн болон продакшн орчин яг ижил байлгах явдал юм. Бэкэнд нь `backend/Dockerfile`-аар тодорхойлогдсон дүрс ашиглана. Node.js 18 Alpine суурь дүрс сонгосноор эцсийн дүрсний хэмжээ жижиг байж, татах болон байршуулах хугацааг хурдасгана.

Эх код 3.1: backend/Dockerfile — Node.js 18 Alpine суурьтай бэкэндийн дүрс

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

EC2 instance дээр Docker суулгасны дараа ECR-аас дүрс татан контейнер ажиллуулна. `-restart unless-stopped` тугаар контейнер дахин эхлүүлэгддэг тул EC2 дахин асах үед бэкэнд автоматаар ажиллана. `.env` файл нь GROQ_API_KEY, EMAIL_USER, EMAIL_PASS болон Firebase тохиргооны нууц мэдээллийг хадгалдаг бөгөөд Git-д оруулдаггүй. Дүрслэлийн архитектурыг доорх диаграммаар харуулав.



Зураг 3.1: Үүлэн байршуулалтын архитектур — AWS EC2, Firebase, CI/CD

Эх: Зохиогчийн боловсруулалт

3.1.2 GitHub Actions CI/CD хоолой

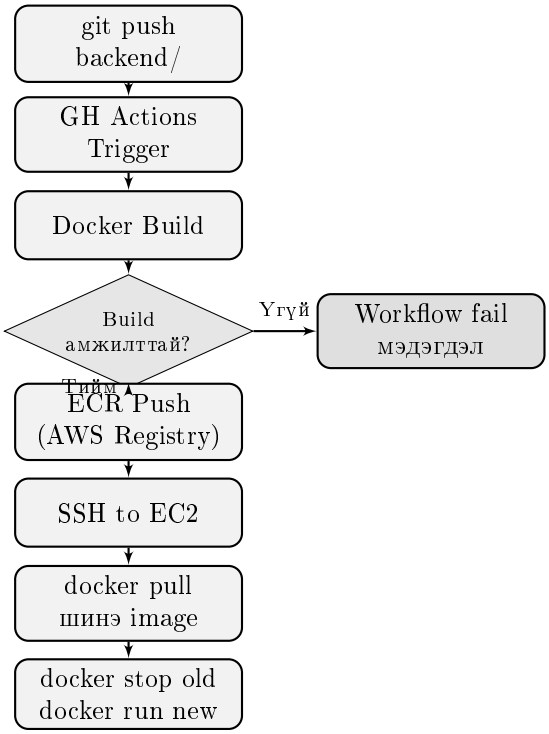
GitHub Actions-ийн CI/CD хоолой нь хөгжүүлэгч `backend/` хавтаст өөрчлөлт push хийх бүрт автоматаар ажилладаг. Нийт байршуулалт 3–4 минут болдог бөгөөд хүний гараар сервер дээр нэвтрэх шаардлага байхгүй болдог. Workflow нь 6 алхамтай: checkout, AWS нэвтрэлт тохируулах, ECR нэвтрэх, Docker build хийж ECR-д push хийх, EC2-д SSH-аар холбогдон шинэ дүрс татаж контейнер солих.

Эх код 3.2: GitHub Actions — автомат байршуулалтын workflow (.github/workflows/deploy.yml)

```
name: Deploy Backend
on:
  push:
    paths: ['backend/**']
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id:   ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ap-southeast-1
      - name: Login and push to ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2
      - name: Build and push Docker image
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
        run: |
          docker build -t $ECR_REGISTRY/dz-zaal-backend:latest ./backend
          docker push $ECR_REGISTRY/dz-zaal-backend:latest
      - name: Deploy to EC2
        env:
          EC2_HOST: ${{ secrets.EC2_HOST }}
          EC2_SSH_KEY: ${{ secrets.EC2_SSH_KEY }}
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
        run: |
          printf '%s\n' "$EC2_SSH_KEY" | tr -d '\r' > /tmp/deploy_key
          chmod 600 /tmp/deploy_key
          ssh -i /tmp/deploy_key -o StrictHostKeyChecking=no \
            ec2-user@$EC2_HOST "
              docker pull $ECR_REGISTRY/dz-zaal-backend:latest
              docker stop dz-zaal-backend || true
              docker rm  dz-zaal-backend || true
              docker run -d --name dz-zaal-backend \
                --restart unless-stopped \
                --env-file ~/backend/.env \
```

```
                -p 3000:3000 \
                $ECR_REGISTRY/dz-zaal-backend:latest"
```

CI/CD хоолойн дараалыг доорх диаграммаар харуулав. Git push-ийн дараа Docker build амжилттай болсон тохиолдолд ECR push болж, EC2-д автоматаар байршуулагддаг.



Зураг 3.2: GitHub Actions CI/CD хоолойн дэлгэрэнгүй дараалын диаграмм
Эх: Зохиогчийн боловсруулалт

3.2 Туршилт ба үнэлгээ

3.2.1 Функциональ туршилт

Туршилт нь 2-р бүлгийн 2.1 хэсэгт тодорхойлсон функциональ шаардлага бүрийг баталгаажуулах зорилготой. Туршилтын орчин: Android 13 (Samsung Galaxy A54), iOS 16 (iPhone 13), AWS EC2 t3.micro (ap-southeast-1), Firebase Cloud Firestore (us-central1), 4G LTE болон Wi-Fi сүлжээ.

Апп-ын үндсэн функц болгоныг бодит хэрэглэгчийн нөхцөлд шалгасан бөгөөд тест бүр нь шаардлагын аль хэсэгтэй холбогдохыг тусгав. Давхардсан захиалгыг шалгахдаа хоёр хэрэглэгч яг нэгэн зэрэг ижил цагт захиалах нөхцөлийг тусгайлан туршиж, Firestore-ийн transaction механизм зөв ажилласныг баталгаажуулсан.

Хүснэгт 3.1: Функциональ туршилтын тохиолдлууд

#	**Шаардлага**	**Туршилтын үйлдэл**	**Хүлээгдэж буй үр дүн**	**Дүгнэлт**
1	Email бүртгэл, баталгаажуулалт	Нэр, email, нууц үг оруулж бүртгүүлэх	Баталгаажуулах код email-д ирэх, `emailVerified:true` болох	**Тэнцсэн**
2	Заалны жагсаалт харах	Нүүр дэлгэц нээх	5 байгууламж, үнэ, байршил зөв гарах	**Тэнцсэн**
3	Цагийн хуваарь харах	Заал, огноо сонгох	16 цагийн үе (08:00–24:00) харагдах	**Тэнцсэн**
4	Цаг захиалах, DBK код авах	Заал, огноо, цаг сонгож баталгаажуулах	DBK-xxxxx кодтой захиалга үүсэх	**Тэнцсэн**
5	Давхардал хориглох	Захиалагдсан цагт дахин захиалах	“Уг цаг авагдсан байна” алдаа гарах	**Тэнцсэн**
6	Захиалга цуцлах	Идэвхтэй захиалгыг цуцлах	Статус `cancelled` болж шинэчлэгдэх	**Тэнцсэн**
7	Захиалгын түүх харах	Захиалгууд tab руу орох	Захиалга статусаар шүүгдэж харагдах	**Тэнцсэн**
8	Локал мэдэгдэл авах	Захиалга хийж хүлээх	Захиалгаас 1 цагийн өмнө мэдэгдэл ирэх	**Тэнцсэн**
9	AI чатбот	“Ямар заалнууд байна вэ?” асуух	Монгол хэлээр заалнуудын жагсаалт ирэх	**Тэнцсэн**
10	Тогтмол захиалга	fixed_bookings цагийг шалгах	Тэр цагийн үе хаагдсан байдлаар харагдах	**Тэнцсэн**

Эх: Зохиогчийн боловсруулалт

Системийн гурван бүрэлдэхүүн хэсгийн хоорондын нэгтгэлийн туршилтыг дараах хүснэгтэд харуулав.

3 дугаар тохиолдолд GitHub Actions-ийн `ssh-action` нь CRLF мөрийн төгсгөлтэй SSH түлхүүрт алдаа гаргаж байсан тул `printf` командыг ашиглан засварлав. Хожим CI/CD хоолой тасралтгүй ажиллаж байгааг 5 дараалсан push-ээр баталгаажуулсан.

3.2.2 Гүйцэтгэлийн хэмжилт ба хэрэглэхэд хялбар байдал

Системийн үндсэн үйлдэл бүрийн хариу цагийг 10 удаа давтан хэмжиж дунджийг гаргасан. Туршилтыг 4G LTE сүлжээн дээр хийсэн бөгөөд хэмжилтийн бүх үзүүлэлт 2.1 хэсэгт тавьсан функциональ бус шаардлагын хязгаарыг хангасан.

Апп анх ачаалах үед Firebase SDK болон локал мэдэгдлийн инициализаци давхар явагддаг тул 2.3 секунд зарцуулагдсан. Энэ нь шаардлагын 3.0 секундын доор

Хүснэгт 3.2: Нэгтгэлийн туршилтын үр дүн

#	**Тайлбар**	**Холбоос**	**Үр дүн**
1	Захиалга үүсгэхэд API дуудаж Firestore-д хадгалагдах	Flutter→EC2→Firestore	**Тэнцсэн**
2	Email бүртгэлийн дараа баталгаажуулах код илгээгдэх	Flutter→Firebase Auth→EC2→Gmail	**Тэнцсэн**
3	Backend-д push хийхэд Docker build, ECR push, EC2 deploy автоматаар явагдах	GitHub→ECR→EC2	**Тэнцсэн**
4	AI чатботын хүсэлт EC2 дамжин Groq API-д хүрч хариулт ирэх	Flutter→EC2→Groq	**Тэнцсэн**

Эх: Зохиогчийн боловсруулалт

Хүснэгт 3.3: Системийн хариу цагийн хэмжилт (секунд) болон шаардлагатай харьцуулалт

Хэмжигдэхүүн	**Дундаж**	**Бага**	**Их**	**Шаардлага**
Апп анх ачаалах	2.3	2.1	2.8	< 3.0 с
Firestore хүсэлт	0.6	0.4	0.9	< 2.0 с
EC2 API хариулт	0.8	0.6	1.2	< 2.0 с
Захиалга үүсгэх	1.1	0.9	1.5	< 2.0 с
Groq Llama хариулт	0.9	0.7	1.3	< 5.0 с
Claude Haiku хариулт	1.4	1.1	2.0	< 5.0 с

Эх: Зохиогчийн боловсруулалт

байна. 15 минут тутмын cron ажлыг шалгахын тулд дуусах цаг нь өнгөрсөн захиалга Firestore-д гараар бүртгэн, ажлын дараагийн гүйцэтгэлийг хүлээсэн. 15 минутын дараа захиалгын статус `upcoming`-аас `completed` болж шинэчлэгдсэн болохыг Firebase Console дээр баталгаажуулсан.

ӨТДС-ийн 5 оюутан болон Даланзадгад хотын 3 иргэн нийт 8 хүнд аппыг танилцуулж 5 даалгавар гүйцэтгүүлсэн: email бүртгүүлэх, заал хайх, захиалга хийх, захиалга харах, AI чатботоос асуух. Үр дүнг доорх хүснэгтэд харуулав.

Дизайн, навигаци, захиалгын процесс, мэдэгдэл, AI чатботын ашигтай байдлыг 1–5 оноогоор үнэлүүлсэн дүнд нийт дундаж оноо **4.3/5.0** гарсан. Захиалгын процессийн оноо 4.1 харьцангуй бага байсан нь "Захиалах" товчийг олоход бага зэрэг хугацаа зарцуулагдсантай холбоотой. Цаашид товчны харагдацыг тодруулах замаар сайжруулах боломжтой. Даланзадгад хотын иргэдийн бүгд захиалга хийх, цуцлах үйлдлийг хугацаандаа зөв хийсэн нь систем орон нутгийн хэрэглэгчдэд тохирсон болохыг харуулсан.

Аюулгүй байдлын шалгалтын хувьд Firebase Auth-ийн хамгаалалт (хэт олон оролдлого хориглох, 10 минутын код хугацаа), API түлхүүрийн нууцлал (GitHub Secrets + EC2 `.env`), Firestore Security Rules (`bookings`-д зөвхөн нэвтэрсэн хэрэглэгч), өгөгдлийн тусгаарлалт (хэрэглэгч зөвхөн өөрийн UID-тэй захиалгыг харах) зэрэг 5 шалгуурын 4 нь бүрэн хангасан. Нэг шалгуур (HTTP → HTTPS) хэсэгчлэн хангасан бөгөөд цаашдын хөгжлийн шатанд SSL нэмэх шаардлагатай.

3.2.3 Туршилтын нийт дүн

Нийт туршилтын тохиолдлын хамрах хүрээг доорх хүснэгтэд нэгтгэн харуулав.

Функциональ туршилт нь 2.1 хэсэгт тодорхойлсон шаардлага бүрийг тусдаа тохиолдлоор шалгасан бөгөөд бүгд амжилттай давсан. Гүйцэтгэлийн хэмжилтийн

Хүснэгт 3.4: Хэрэглэхэд хялбар байдлын үнэлгээний үр дүн

Даалгавар	**Амжилттай**	**Дундаж хугацаа**	**Тэмдэглэл**
Email бүртгэл/нэвтрэх	8/8	55 сек	Баталгаажуулах код 3–8 сек
Заал хайх	8/8	20 сек	Нүүр дэлгэц шууд ачаалагдсан
Захиалга хийх	7/8	1 мин 15 сек	1 хүн "Захиалах" товчийг эхэнд олоогүй
Захиалга харах	8/8	10 сек	Маш хялбар
AI чатбот	8/8	35 сек	Монгол хэлээр асуух хялбар

Эх: Зохиогчийн боловсруулалт

Хүснэгт 3.5: Туршилтын нийт үр дүн — шаардлагатай харьцуулалт

Туршилтын төрөл	**Нийт**	**Тэнцсэн**	**Хувь**
Функциональ (шаардлага бүрт нэг тохиолдол)	10	10	100%
Нэгтгэлийн (бүрэлдэхүүн хоорондын)	4	4	100%
Гүйцэтгэлийн (хариу цагийн хязгаар)	6	6	100%
Нийт	**20**	**20**	**100%**

Эх: Зохиогчийн боловсруулалт

бүх үзүүлэлт функциональ бус шаардлагын хязгаарыг хангасан. Хэрэглэхэд хялбар байдлын нийт дундаж оноо 4.3/5.0 нь систем нь орон нутгийн иргэдэд тохиромжтой болохоо харуулсан. Туршилтын явцад нэг функциональ алдаа байгаагүй хэдий ч хоёр сайжруулах санал гарсан: захиалгын товчны харагдацыг тодруулах, EC2-ийн холболтод SSL нэмэх. Эдгээрийг цаашдын хөгжлийн шатанд оруулах зорилтот болгосон.

Дүгнэлт

Энэхүү бакалаврын төгсөлтийн ажлын хүрээнд Өмнөговь аймгийн Даланзадгад хотын таван спорт байгууламж — Өмнөговь аймгийн Спорт цогцолбор (спорт цогцолбор) болон Говийн Арена, 1-р болон 2-р хорооны Спорт заал, Даланзадгад Волейболын клуб заал (хот айлын заалнууд) — -д зориулсан **үүлэн орчинд суурилсан цаг захиалгын мобайл аппликейшн**-ийг Flutter, Node.js/Express, Firebase, Docker, AWS болон AI технологиудыг хосолсон орчин үеийн бүрэн стекэд хөгжүүлж, дараахи зорилтуудыг хэрэгжүүлсэн:

1. **Асуудлын судалгаа ба шаардлагын шинжилгээ.** Даланзадгад хотын иргэдийн тулгамдаж буй таван үндсэн асуудлыг тогтоож — утсаар захиалах, биечлэн очих, цагийн мэдээлэл ил тод бус, цуцлах боломжгүй, сануулга байхгүй гэсэн — тус бүрд нь цахим шийдлийг тодорхойлсон. Функциональ болон функциональ бус шаардлагуудыг нарийвчлан баталгаажуулсан.

2. **Flutter мобайл аппликейшн.** Material Design 3 стандартад нийцсэн, дулаан говийн цайвар дизайн (бежийн дэвсгэр `#EDE7DA`, наранжин `#D4700F`, Montserrat фонт) бүхий мобайл аппликейшнийг 4 tab-тай доод навигацитай хэрэгжүүлсэн. StatefulWidget + setState суурилсан төлвийн удирдлага нэмэлт dependency байхгүйгээр тогтвортой ажилласан. Апп нь Android болон iOS платформд нэгэн зэрэг ажилладаг.

3. **Firebase үйлчилгээ ба email нэвтрэлт.** Firebase Authentication-ийн email/нууц үгийн нэвтрэлтийг Node.js бэкэнд дамжуулан Nodemailer (Gmail SMTP)-ийн 6 оронтой баталгаажуулах кодтой хосолсон. Кодын хугацаа 10 минут бөгөөд амжилтгүй тохиолдолд Firebase rollback хэрэгжинэ. Cloud Firestore-д `bookings`, `users`, `fixed_bookings`, `email_verifications` гэсэн дөрвөн collection тодорхойлж, DBK-xxxxx захиалгын кодын систем хэрэгжүүлсэн.

4. **Node.js/Express REST API бэкэнд.** 8 endpoint бүхий RESTful API хэрэгжүүлж, email баталгаажуулалт, захиалгын менежмент, цагийн хуваарь, AI чат зэрэг функцийг хангасан. `lib/config.dart` файлаар хөгжүүлэлтийн болон продакшн орчны URL-ийг нэг газраас удирддаг.

5. **Үүлэн байршуулалт: Docker + AWS EC2.** Бэкэнд сервер нь Docker контейнержуулалтаар **AWS EC2 (ap-southeast-1, Сингапур)** дээр бодитоор ажиллаж байна. AWS ECR-д Docker image хадгалж, EC2 дээр ажиллуулдаг энэ cloud-native архитектур нь системийн тасралтгүй байдал, хэмжээний уян хатан байдлыг хангадаг.

6. **GitHub Actions CI/CD автоматжуулалт.** `backend/` хавтаст өөрчлөлт push хийхэд автоматаар Docker image build → AWS ECR push → EC2 SSH deploy гэсэн дараалаар 3–4 минутын дотор байршуулалт хийгддэг. 5 дараалсан push-д туршиж, тасралтгүй ажилласныг баталгаажуулсан.

7. **Давхар горимын AI чатбот.** Anthropic Claude Haiku 4.5-г Firebase Cloud Functions дамжуулан (үндсэн горим) болон Groq Llama 3.1-8b-г Node.js бэкэнд дамжуулан (нөөц горим) ажиллуулах нөөцлөлтийн архитектур нэвтрүүлсэн. Энэ нь AI үйлчилгээний найдвартай байдлыг нэмэгдүүлнэ.

8. **Туршилтын үр дүн.** Нийт 20 туршилтын тохиолдол бүгд амжилттай давсан: функциональ 10/10, нэгтгэлийн 4/4, гүйцэтгэлийн 6/6. 8 хэрэглэгч дээр

хийсэн хэрэглэхэд хялбар байдлын үнэлгээний нийт дундаж оноо **4.3/5.0** болсон. Апп 2.3 секундын дотор ачаалж, Firestore хүсэлт 0.6 с, EC2 API 0.8 с хариулдаг болохыг хэмжиж баталгаажуулсан.

Туршилтын нэгтгэсэн дүн:

Хүснэгт 3.6: Зорилтуудын биелэлт

#	**Зорилт**	**Биелэлт**
1	Асуудлын судалгаа, шаардлагын шинжилгээ	**Бүрэн**
2	UML диаграммууд (Use Case, Activity, Sequence)	**Бүрэн**
3	Flutter мобайл аппликейшн (Android/iOS)	**Бүрэн**
4	Firebase Auth + Firestore + Cloud Functions	**Бүрэн**
5	Node.js/Express REST API	**Бүрэн**
6	Docker + AWS EC2 үүлэн байршуулалт	**Бүрэн**
7	GitHub Actions CI/CD	**Бүрэн**
8	AI чатбот (Claude Haiku + Groq Llama)	**Бүрэн**
9	Функциональ болон гүйцэтгэлийн туршилт	**Бүрэн**

Цаашдын хөгжлийн чиглэл:

- **Онлайн төлбөр** — QPay, SocialPay зэрэг Монголын төлбөрийн системүүдтэй нэгтгэх;
- **Администраторын дэлгэц** — Спорт заалны ажилтнуудад зориулсан хуваарь удирдах вэб дэлгэц хөгжүүлэх;
- **HTTPS/SSL** — EC2 болон Flutter апп хоорондын холболтод SSL нэмж аюулгүй байдлыг сайжруулах;
- **Firebase Storage** — Спорт заалнуудын зургийг Firestore-тэй уялдуулан хадгалах;
- **GPS байршил** — Хэрэглэгчид хамгийн ойр заалыг харуулах функц;
- **Статистик самбар** — Захиалгын дүн шинжилгээ, ачаалалтай цагийн мэдээлэл;
- **Өргөтгөл** — Даланзадгад хотоос гадна Өмнөговь аймгийн бусад сумдын спорт байгууламжид өргөтгөх.

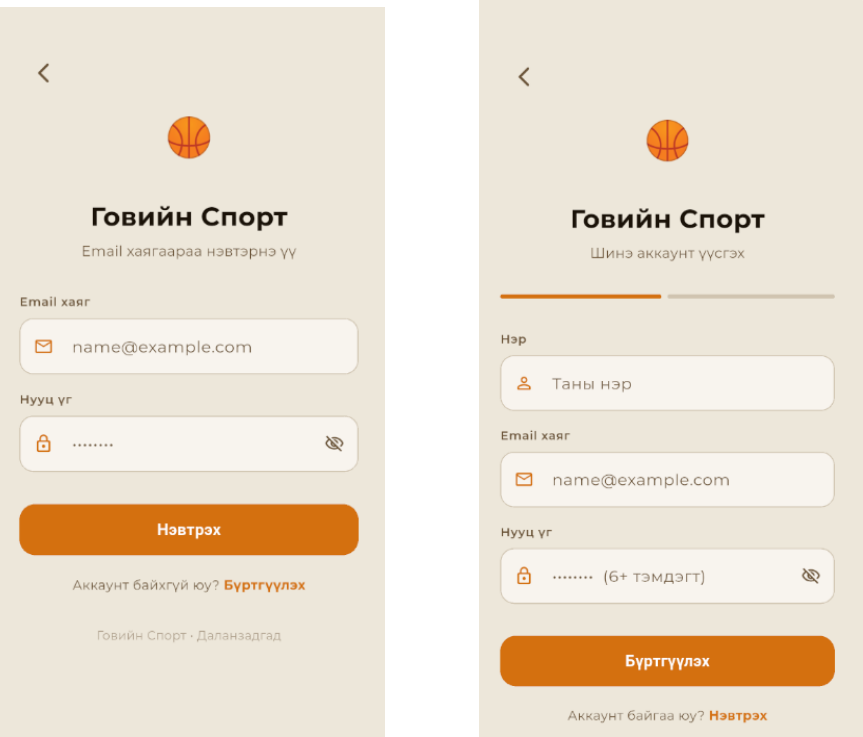
Энэхүү ажил нь Монгол улсын орон нутгийн хотуудад ухаалаг цахим үйлчилгээ нэвтрүүлэх боломжийг тодорхой харуулж, Flutter болон Firebase-г Node.js/Express REST API, Docker/AWS үүлэн дэд бүтэц, GitHub Actions CI/CD автоматжуулалт, AI чатбот үйлчилгээтэй хосолсон бүрэн стекийн туршлагыг цуглуулсан чухал бүтээл болсон гэж дүгнэж байна. Хөгжүүлсэн систем нь бодитоор үүлэн орчинд (AWS EC2) ажиллаж байгаа нь ирээдүйн орон нутгийн цахим үйлчилгээний системүүдэд загвар болж чадна.

ХАВСРАЛТ А

Аппликейшний дэлгэцийн зураглал

Энэ хавсралтад ДЗ-Заал мобайл аппликейшний үндсэн дэлгэцүүдийн screenshot-уудыг харуулав.

А.1 Нэвтрэлт ба бүртгэлийн дэлгэц



Зураг А.1: Нэвтрэх дэлгэц (зүүн) ба бүртгэлийн дэлгэц (баруун)

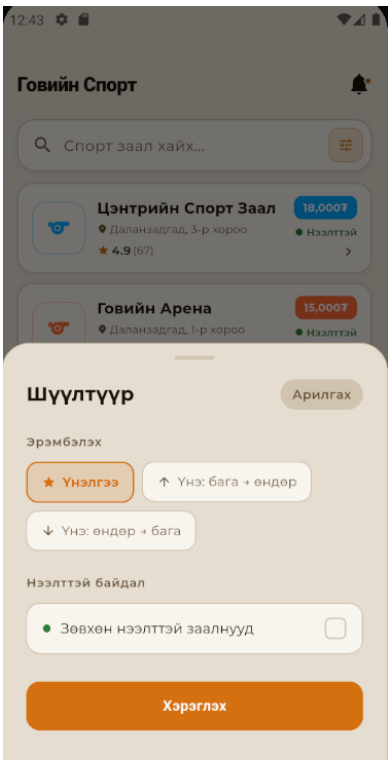
Email хаяг болон нууц үгээр нэвтрэх дэлгэц нь цайвар бежийн дэвсгэр (`#EDE7DA`) дээр Montserrat фонтоор загварчлагдсан. Бүртгэлийн дэлгэцэд нэр, email, нууц үг оруулах талбарууд байх бөгөөд “Бүртгүүлэх” товч дарахад Nodemailer-ийн тусламжтайгаар 6 оронтой баталгаажуулах код илгээгдэнэ.

А.2 Нүүр дэлгэц — Спорт заалнуудын жагсаалт

Нүүр дэлгэц нь апп нэвтрэлтийн дараа харагдах анхны дэлгэц юм. Даланзадгадын таван спорт заал картан хэлбэрээр жагсаагдаж, заал бүрийн нэр, байршил, нэг цагийн үнийг харуулна. “Захиалах” товч дарахад цагийн хуваарийн дэлгэц нээгдэнэ.

А.3 Цагийн хуваарь ба захиалгын дэлгэц

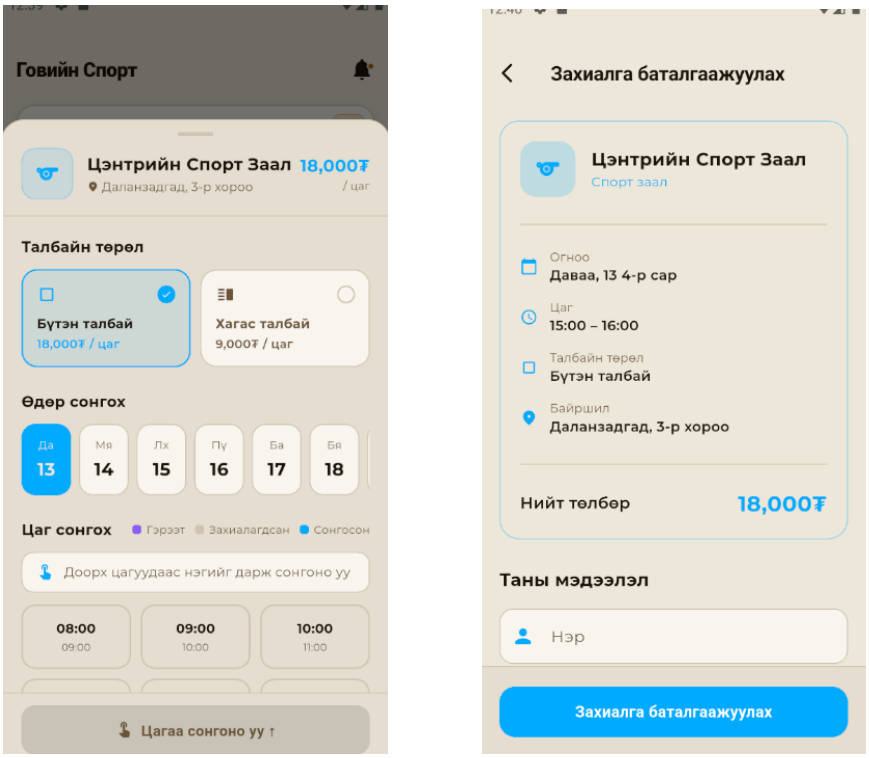
Цагийн хуваарийн дэлгэцэд сонгосон заал болон огнооны 08:00–00:00 хооронд 16 цагийн үе харагдана. Боломжтой цаг ногоон, захиалагдсан цаг улаан, байгууллагын тогтмол захиалга нил ягаан өнгөөр тэмдэглэгдэнэ. Захиалга баталгаажуулах дэлгэцэд DBK-xxxxx форматын захиалгын код харагдана.



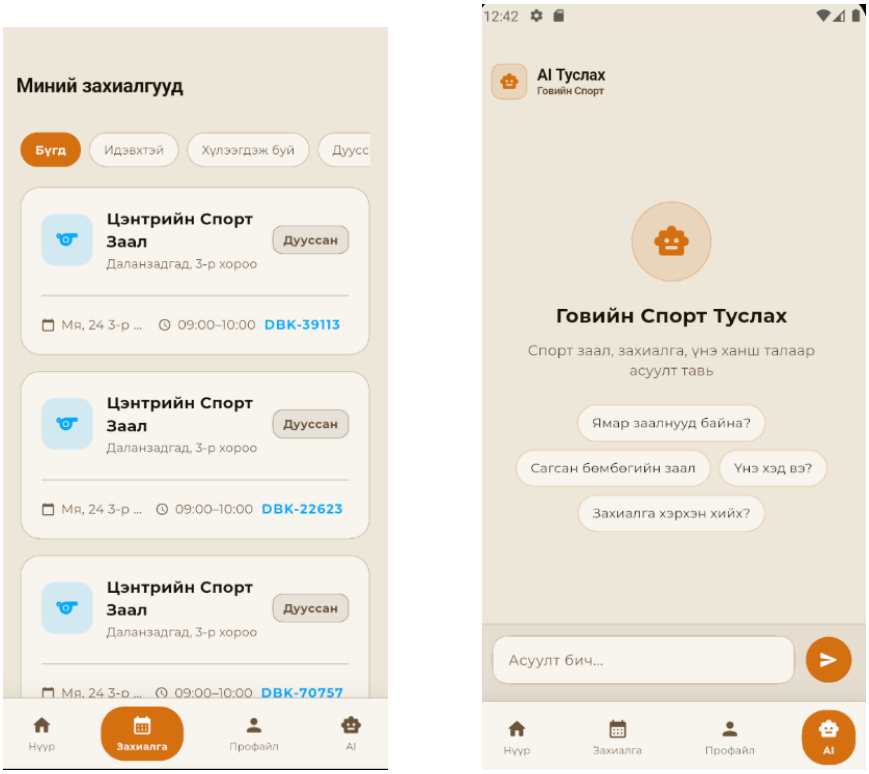
Зураг А.2: Нүүр дэлгэц — таван спорт заалны жагсаалт

А.4 Захиалгын жагсаалт ба AI чатботын дэлгэц

Захиалгын дэлгэцэд хэрэглэгчийн бүх захиалга статусаар (идэвхтэй, дууссан, цуцлагдсан) шүүгдэж харагдана. AI чатботын дэлгэцэд Anthropic Claude Haiku болон Groq Llama загваруудаар дамжуулан спорт заалтай холбоотой асуулт асуух боломжтой.



Зураг А.3: Цагийн хуваарийн дэлгэц (зүүн) ба захиалга баталгаажуулах дэлгэц (баруун)



Зураг А.4: Захиалгын жагсаалтын дэлгэц (зүүн) ба AI чатботын дэлгэц (баруун)

ХАВСРАЛТ В

Үндсэн тохиргооны файлууд

Энэ хавсралтад системийн байршуулалт болон автоматжуулалттай холбоотой үндсэн тохиргооны файлуудыг оруулав.

Б.1 Dockerfile — Node.js/Express бэкэнд

Эх код В.1: backend/Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Node.js 18 LTS-ийн alpine суурь дүрс ашигласнаар Docker дүрсний хэмжээ хамгийн бага байна. `npm ci -only=production` нь зөвхөн production хамаарлуудыг суулгаж, хөгжүүлэлтийн хамаарлуудыг оруулахгүй байлгана.

Б.2 GitHub Actions CI/CD Workflow

Эх код В.2: .github/workflows/deploy.yml

```
name: Deploy to AWS EC2

on:
  push:
    branches: [main]
    paths:
      - 'backend/**'

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ap-southeast-1

      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v1

      - name: Build and push Docker image
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
```

```
        IMAGE_TAG: ${{ github.sha }}
      run: |
        docker build \
          -t $ECR_REGISTRY/dz-zaal-backend:$IMAGE_TAG \
          -t $ECR_REGISTRY/dz-zaal-backend:latest \
          ./backend
        docker push $ECR_REGISTRY/dz-zaal-backend:$IMAGE_TAG
        docker push $ECR_REGISTRY/dz-zaal-backend:latest

    - name: Deploy to EC2
      uses: appleboy/ssh-action@master
      with:
        host: ${{ secrets.EC2_HOST }}
        username: ubuntu
        key: ${{ secrets.EC2_SSH_KEY }}
        script: |
          aws ecr get-login-password --region ap-southeast-1 | \
            docker login --username AWS --password-stdin \
            ${{ secrets.ECR_REGISTRY }}
          docker pull ${{ secrets.ECR_REGISTRY }}/dz-zaal-backend:latest
          docker stop dz-zaal-backend || true
          docker rm dz-zaal-backend || true
          docker run -d --name dz-zaal-backend \
            -p 3000:3000 \
            --env-file /home/ubuntu/.env \
            ${{ secrets.ECR_REGISTRY }}/dz-zaal-backend:latest
```

Workflow нь `backend/` хавтаст өөрчлөлт push хийх үед автоматаар ажиллана. AWS нэвтрэлтийн мэдээлэл (access key, SSH түлхүүр) нь GitHub Secrets-д хадгалагдаж кодод задрахгүй.

Б.3 Firebase Firestore Security Rules

Эх код В.3: firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /users/{userId} {
      allow read, write: if request.auth != null
                         && request.auth.uid == userId;
    }

    match /bookings/{bookingId} {
      allow read: if request.auth != null
                  && resource.data.userId == request.auth.uid;
      allow create: if request.auth != null;
      allow update, delete: if request.auth != null
                            && resource.data.userId == request.auth.uid;
    }
```

```

    match /fixed_bookings/{docId} {
      allow read: if request.auth != null;
      allow write: if false;
    }

    match /email_verifications/{docId} {
      allow read, write: if false;
    }
  }
}
```

Firestore аюулгүй байдлын дүрэм нь хэрэглэгч зөвхөн өөрийн захиалгыг унших, засах эрхтэй болохыг баталгаажуулна. `email_verifications` болон `fixed_bookings` цуглуулгуудад клиент тал бичих боломжгүй.

Ном зүй

[1] Statista Research Department. *Mobile Applications — Worldwide Revenue Forecast 2024–2028*. 2024. URL: https://www.statista.com/outlook/amo/app/worldwide.

[2] DAZO LLC. *eZaal — Спортын байгууламжийн захиалгын платформ*. Google Play, 10,000+ татагдсан, хувилбар 2.2.1, 2025-06-01. 2025. URL: https://play.google.com/store/apps/details?id=mn.e.zaal.

[3] A. Kassim, M. Rahman **and** Z. Hasan. "Development of Sports Facility Booking Mobile Application Using Flutter and Firebase". **in***International Journal of Advanced Computer Science and Applications*: 13.4 (2022), **pages** 412–420.

[4] Амарсайханы Билгүүн. "Ухаалаг гар утасны аппликейшн хөгжүүлэлт". Бакалаврын төгсөлтийн ажил. ШУТИС-МХТС, 2022.

[5] Т.Амартүвшин **and** Б.Зоригтбаатар. *Мобайл аппликейшн хөгжүүлэлт*. Улаанбаатар: ШУТИС, Хэвлэлийн газар, 2020.

[6] Google LLC. *Flutter — Build apps for any screen*. 2024. URL: https://flutter.dev/docs.

[7] Google LLC. *Material Design 3*. 2024. URL: https://m3.material.io.

[8] Google LLC. *Dart Programming Language*. 2024. URL: https://dart.dev/guides.

[9] Google LLC. *Firebase — App Development Platform*. 2024. URL: https://firebase.google.com/docs.

[10] Google LLC. *Cloud Firestore — Firebase*. 2024. URL: https://firebase.google.com/docs/firestore.

[11] Google LLC. *Authenticate Users — Firebase Authentication*. 2024. URL: https://firebase.google.com/docs/auth.

[12] Michael Bui. *flutter_local_notifications — A cross platform plugin for displaying local notifications*. 2024. URL: https://pub.dev/packages/flutter_local_notifications.

[13] OpenJS Foundation. *Node.js — JavaScript runtime built on Chrome's V8 engine*. 2024. URL: https://nodejs.org/en/docs.

[14] OpenJS Foundation. *Express — Fast, unopinionated, minimalist web framework for Node.js*. 2024. URL: https://expressjs.com.

[15] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000. URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm.

[16] Docker Inc. *Docker Documentation — Develop faster. Run anywhere*. 2024. URL: https://docs.docker.com.

[17] Amazon Web Services. *Amazon EC2 — Secure and resizable compute capacity in the cloud*. 2024. URL: https://aws.amazon.com/ec2.

[18] Amazon Web Services. *Amazon Elastic Container Registry — Share and deploy container software*. 2024. URL: https://aws.amazon.com/ecr.

[19] GitHub Inc. *GitHub Actions — Automate your workflow from idea to production*. 2024. URL: https://docs.github.com/en/actions.

[20] Anthropic. *Claude API Documentation — Anthropic*. 2024. URL: https://docs.anthropic.com.

[21] Groq Inc. *Groq API Documentation — Fast AI Inference*. 2024. URL: https://console.groq.com/docs.